SOFTOMATIC
TECHNOLOGY SERVICES

EXAMATE · SYSTEM DESIGN · ELITE + BATCH 1

Lesson 8 · Choosing a Datastore: Relational, Key-value, Document, Wide-column

"SQL or NoSQL?" is the wrong first question. This lesson teaches the right one, "what are the access patterns?", through one story: Discord's chat messages, which outgrew one database, then a second, and moved again, while the query never changed.

MODULE
System Design · Part A
Foundations

TIME
About 80 minutes
plus 45 minutes of practice problems

YOU NEED
Lessons 3, 6 and 7
access patterns, Redis, shard keys

YOUR ROUTE TODAY

01 Discord's messages outgrow one replica set
02 Four families, one map
03 Access patterns pick the store
04 Key-value, document, wide-column
05 **One app, many stores**

LESSON 8 AT A GLANCE

What you will be able to do

Four families
Relational, key-value, document, wide-column: what each is good and bad at.

Access patterns first
A one-line-per-query list that picks the store for you.

Key-value
Redis vs DynamoDB, capacity units, and the per-partition limit.

Documents
Embed or reference, and the 16 MiB trap.

Wide-column
Partition and clustering keys, the write path, tombstones.

Polyglot
One product, many stores, one source of truth per fact.

Subject / module	System Design / Part A: Foundations
Difficulty	Intermediate. You need Lesson 3's access patterns and transactions, Lesson 4's arithmetic, Lesson 6's Redis, and Lesson 7's shard keys and hot keys.
Prerequisites	Lessons 1 to 7. Lesson 3 said "start from access patterns, not from tables". Lesson 4 sized sho.rt at 7,716 redirects per second at peak, about 77 creations per second, 6 billion links (up to 3 TB) in five years. Lesson 6 put Redis in front as a cache. Lesson 7 sharded the click table (about 333.5 million rows a day, 24.3 TB for two years) by hash of link_code and met hot keys.
Next lesson	Lesson 9: CAP, consistency models and quorums
Companies covered	Zomato, Swiggy, Uber, Amazon (dated reports); Infosys (Candidate-reported). Everything else is Pattern-only.
Commands and configs	DynamoDB capacity units and per-partition limits; a MongoDB menu document; a Cassandra CREATE TABLE with partition and clustering keys; Redis AOF appendfsync everysec; PostgreSQL jsonb with a GIN index.
Checklist	☐ what each store type is good and bad at ☐ access patterns decide the store ☐ key-value (Redis, DynamoDB) use cases ☐ document stores (MongoDB) use cases ☐ wide-column (Cassandra) use cases ☐ polyglot persistence in one product

Numbers you bring from earlier lessons

7,716/s
sho.rt redirects at peak (Lesson 4)

77/s
link creations at peak (Lesson 4)

772/s
DB reads after a 90% cache hit (Lesson 6)

3 TB
6 billion links in five years (Lesson 4)

24.3 TB
two years of clicks (Lesson 7)

20,000/s
clicks on one viral link (Lessons 6, 7)

Lesson 3 taught one relational database well. Lesson 7 copied and split it. This lesson asks the question before both: **should this data be relational at all?**

HOW TO READ THIS LESSON

Outcomes and evidence labels

What you will be able to do after this lesson:

- Name the four store families (relational, key-value, document, wide-column), say what each one is good and bad at, and give one real product that uses each.
- Write an access-pattern list for a feature (query, frequency, shape, consistency need) and use it to pick a store, instead of picking by fashion.
- Size a DynamoDB table in read and write capacity units, and spot a hot partition against the 3,000 RCU / 1,000 WCU per-partition limit.
- Decide between embedding and referencing in a MongoDB document, and catch an unbounded array before it hits the 16 MiB document limit.
- Design a Cassandra table from its query: choose the partition key and clustering key, bound the partition size, and explain the write path and tombstones.
- Draw a polyglot data layer for a food-delivery app, justify every store with an access pattern, and say how the stores stay in sync.

SOURCE-CONFIRMED

A dated first-person report naming the company and the question or topic, October 2024 to October 2026, opened in full.

CANDIDATE-REPORTED

Compilations, aggregator listings, undated posts.

PATTERN-ONLY

Commonly asked, no company source, so no company name. Practice problems and quiz questions are original and unlabelled.

AN HONEST NOTE ON THE EVIDENCE

This topic shows up mostly as "why this database?" about your own project, or as "SQL or NoSQL, and why?" inside a design round. A **Zomato** SDE-1 candidate (July 2025) was asked why they did not use MongoDB and when to use Postgres [S24]; a **Zomato** SDE intern candidate (April 2026) was asked why they used both PostgreSQL and MongoDB in one project [S25]; a **Swiggy** SDE II candidate (interviewed November 2024) was asked the differences, use cases, advantages and disadvantages of SQL and NoSQL [S26]; an **Uber** SDE-2 candidate in Bengaluru (September 2025) had to justify SQL or NoSQL for an Uber Eats homepage [S27]; two **Amazon** SDE-2 candidates (May 2025, January 2026) chose a database for a blogging platform and discussed an inventory service on NoSQL [S28, S29]. No dated report asked specifically about Cassandra's write path or DynamoDB capacity units, so those are Pattern-only.

INTERVIEW MODE: SAY IT LIKE THIS

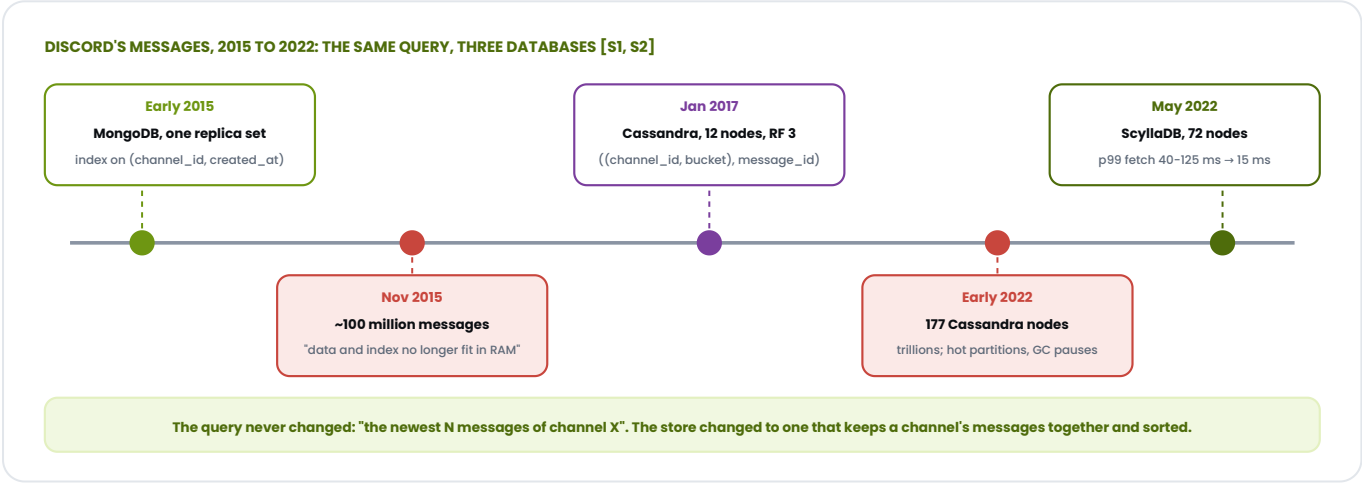
- This lesson covers the four store families and their strengths and weaknesses, how access patterns decide the store, key-value stores (Redis and DynamoDB), document stores (MongoDB), wide-column stores (Cassandra), and polyglot persistence in one product.

DISCORD, 2015 TO 2022

Same query. Three databases.

Early 2015. Discord's first version is built in under two months, and every chat message goes into **one MongoDB replica set**, with a compound index on channel_id and created_at [S1].

November 2015. Discord has about **100 million stored messages**. In the words of its CTO, **"the data and the index could no longer fit in RAM"**, and latencies became unpredictable [S1]. Reads were extremely random and the read-to-write ratio was about **50/50**: a small private server mostly reads recent messages that are rarely in the disk cache; a voice-heavy server sends so few that loading 50 of them can mean many random disk seeks [S1].



PAUSE AND PREDICT

The data is not even complicated, just messages in channels. Why would a **different kind** of database fix this, rather than a bigger machine?

Answer: Discord's main query is always the same shape: "the newest N messages of channel X". A store that keeps one channel's messages **together on disk, already sorted by time**, answers it with one short sequential read, and adds machines without manual re-sharding. Discord moved to **Cassandra** (wide-column), keyed by ((channel_id, bucket), message_id) with 10-day buckets, on 12 nodes in 2017 [S1]. By 2022 the same design held trillions of messages on 177 nodes; Discord then moved it to ScyllaDB on 72 nodes [S2].

~100 million
messages when MongoDB's index outgrew RAM (Nov 2015) [S1]

50/50
read/write ratio, extremely random reads [S1]

12 → 177
Cassandra nodes, 2017 to 2022 [S1, S2]

177 → 72
nodes after moving to ScyllaDB [S2]

WHAT THE STORY IS REALLY ABOUT

Not "MongoDB bad, Cassandra good"

MongoDB was a fine choice for a two-month prototype. The lesson is that **the shape of your most important query, multiplied by your scale, decides the store**. The same Discord team later left Cassandra too, for a Cassandra-compatible store, because of hot partitions and garbage-collection pauses at 177 nodes [S2]. The query never changed.

YEAR	STORE	WHY IT CHANGED
2015	MongoDB, one replica set	Built in two months; fine until data and index outgrew RAM [S1]
2017	Cassandra, 12 nodes, RF 3	Linear scaling, automatic failover, a channel's messages contiguous on disk [S1]
2022	ScyllaDB, 72 nodes (from 177)	Hot partitions, GC pauses, compaction backlog; p99 fetch 40–125 ms → 15 ms [S2]

Discord's own requirements list (2017) reads like this lesson's checklist [S1]. Notice that not one line says "NoSQL":

DISCORD'S REQUIREMENT [S1]	WHERE IT SHOWS UP IN THIS LESSON
Linear scalability, no manual re-sharding	Concept 6 (partition key on a ring) and Lesson 7
Automatic failover	Lesson 7 replication; Concept 6
Predictable performance (alert when API p95 exceeds 80 ms), no reliance on a cache	Concepts 2 and 8
Low maintenance, proven, open source	Concept 9: every store is an on-call cost
Not a blob store	Concept 5: size limits per item

Just for fun. Discord did not "upgrade to NoSQL". It moved from one NoSQL database to another NoSQL database, and later to a third. The word "NoSQL" tells you almost nothing.

TINY CHALLENGE

Discord's 2017 cluster had 12 nodes with replication factor 3. How many copies of each message existed? Answer: 3, on 3 different nodes.

MEMORY HOOK: QUERY X SCALE

The **store** follows the **shape of the hottest query**, times the **scale**. Never the fashion.

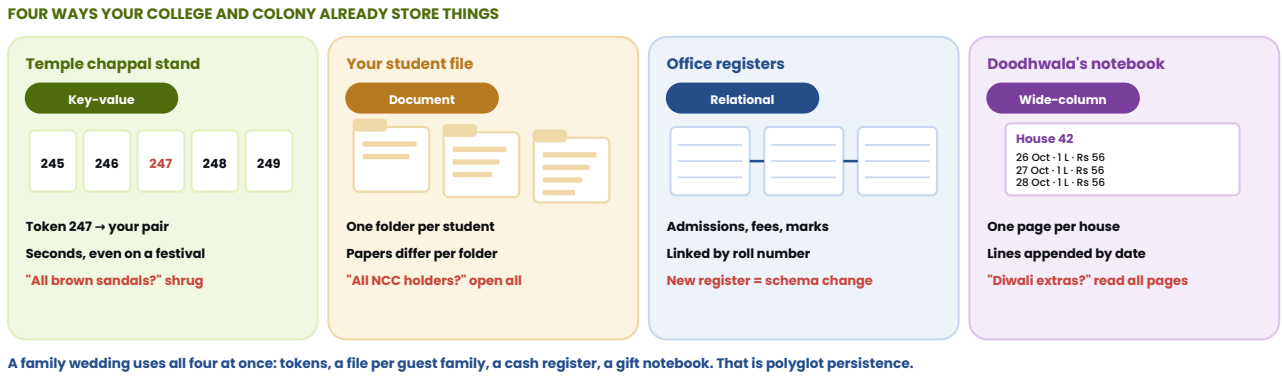
INTERVIEW MODE: SAY IT LIKE THIS

- A datastore is chosen from the access patterns and the scale, not from the shape of the data alone.
- Discord's reads were random but always "latest messages in one channel", so a wide-column store that keeps each channel's messages contiguous and sorted fit better than one replica set whose index no longer fit in memory.

BEFORE THE JARGON

The chappal stand, the student file, the register, the doodhwala

Think of four ways your college and your colony already store information.



- **The temple chappal stand (key-value).** Hand over your chappals, get token 247. Show 247 later and get exactly your pair back, in seconds, even on a festival day. Ask "show me every brown sandal" and the attendant can only shrug: the stand is organised by token, nothing else.
- **Your student file in the office (document).** One folder per student with everything inside: admission form, marksheets, sports certificate, photos. One folder gives the whole picture; two folders need not contain the same papers. "Every student with an NCC certificate" means opening every folder, unless someone keeps a separate list.
- **The office registers (relational).** Admissions, fees and marks registers, linked by roll number. The clerk can answer almost any question by cross-checking registers, and a fee payment is stamped in the receipt book and the fees register together or not at all.
- **The doodhwala's notebook (wide-column).** One page per house; every morning he appends a line: date, litres, rate. "What did House 42 take this month?" Flip to page 42, read the last 30 lines, already in date order. "Which houses took extra milk on Diwali?" means reading every page.
- **The wedding at home (polyglot).** On a family wedding day all four are in use at once. Nobody tries to run the whole wedding from one notebook.

WHAT GOES WRONG

Where the analogy breaks. The chappal attendant cannot expire your token; Redis can (TTL, Lesson 6). Real document stores can index fields inside the folders, so "find all NCC certificates" can be fast if you planned an index. And stores borrow from each other: PostgreSQL stores JSON documents with indexes [S16], DynamoDB has transactions [S4], MongoDB has multi-document transactions [S8]. The families describe the **default shape** of data and queries, not hard walls.

THE WHOLE LESSON IN ONE TABLE

Home and college to store family

AT HOME OR COLLEGE	STORE FAMILY
Chappal stand: token → your pair	Key-value
One folder per student, papers vary	Document
Registers linked by roll number, stamped together	Relational (joins, transactions)
Doodhwala's notebook: page per house, lines by date	Wide-column (partition key, clustering key)
A family wedding using all four at once	Polyglot persistence

One thing, four ways. The same Examate test attempt, stored in each family:

FAMILY	HOW THE ATTEMPT IS STORED	WHAT BECOMES EASY
Key-value	attempt:{id} → one blob	Load or save the whole attempt by id
Document	{ attempt_id, student, answers: […] }	Load the attempt with its answers; varying question types
Relational	attempts table + answers table, joined by attempt_id	"Average score per college per topic" in one SQL query
Wide-column	PRIMARY KEY ((exam_id, student_id), answered_at)	Append every answer as it happens; replay one student's timeline

Just for fun. Every hostel has one student who stores everything in one notebook: timetable, expenses, crush's birthday, DSA notes. That student is a monolith. Lesson 7 showed you what happens to monoliths.

TINY CHALLENGE

In the doodhwala analogy, what is a "hot partition"?
Answer: The page for the hostel mess: 40 litres a day and a line every hour, while most houses get one line a day.

MEMORY HOOK: TOKEN, FOLDER, REGISTER, NOTEBOOK

Token = key-value. **Folder** = document. **Register** = relational. **Notebook** = wide-column.

INTERVIEW MODE: SAY IT LIKE THIS

- A key-value store answers "give me the value for this key" and nothing else. A document store keeps one entity's related data together in a flexible document.
- A relational store keeps normalized tables linked by keys with joins and multi-row transactions. A wide-column store groups rows by a partition key and sorts them by a clustering key, so appends and "latest rows for this key" are cheap.

EVERY WORD, ONE LINE (1 OF 2)

Vocabulary: the families

Terms from earlier lessons (ACID, isolation, index, cache, TTL, replica, shard, shard key, hot key, consistent hashing, eventual consistency) are reused without re-definition.

Datastore
Any system that stores data durably for an application: a database, a key-value store, a search engine.

Relational database (RDBMS)
Tables with a fixed schema linked by keys; SQL; joins and multi-row transactions (Lesson 3).

NoSQL
A loose umbrella for non-relational stores. It names what a store is not, not what it is.

Schema-on-write
The database checks the shape of data when it is written (relational tables).

Schema-on-read
Flexible shapes accepted; the application interprets them when reading (unless validation is added).

Key-value store
Stores values under unique keys; main operations are get, put and delete by key.

Redis
In-memory store with rich types (strings, hashes, sorted sets, streams); cache (Lesson 6) and sometimes primary store [S14, S15].

DynamoDB
AWS's managed, durable key-value and document store; items found by partition key and optional sort key [S3].

Item / attribute
DynamoDB's row and field. Maximum item size 400 KB [S4].

RCU / WCU
1 RCU = one strongly consistent read/s (or two eventual) up to 4 KB; 1 WCU = one write/s up to 1 KB [S6].

GSI
Global secondary index: a DynamoDB index on a different key; reads are eventually consistent [S5].

Document store
Stores self-contained JSON-like documents in collections, with indexes on fields.

MongoDB
A document database; documents are BSON, at most 16 MiB each [S7].

Collection
MongoDB's group of documents, roughly a table.

Embedding / referencing
Related data inside the parent document, or in another document linked by id [S8].

Denormalization
Storing the same fact in more than one place so a read needs no join.

Single-table design
A DynamoDB pattern: several entity types in one table, told apart by key prefixes (Zomato's billing table [S17]).

Conditional write
A write that succeeds only if a condition holds (DynamoDB condition expressions, Cassandra IF): stops overselling.

EVERY WORD, ONE LINE (2 OF 2)

Vocabulary: wide-column and polyglot

Wide-column store
Rows grouped into partitions by a partition key, sorted inside by clustering columns (Cassandra, ScyllaDB, HBase, Bigtable).

Cassandra
Open-source wide-column database; query-driven modelling; partitions placed with a variant of consistent hashing [S9].

Partition key
Decides which nodes hold the data: Lesson 7's shard key, built in.

Clustering key (DynamoDB: sort key)
Orders rows inside one partition, so "latest 50" is one sequential read.

Primary key (Cassandra)
Partition key plus clustering columns; together they identify one row.

Commit log / memtable / SSTable
The write path: append to a log, write to memory, flush to immutable sorted files [S10].

Compaction
Background merging of SSTables: fewer files per read, old versions dropped.

Tombstone
A "this was deleted" marker, removed only later during compaction.

LSM tree
Log-structured merge tree: memtable + SSTables + compaction; makes writes cheap.

Lightweight transaction (LWT)
Cassandra's compare-and-set (IF NOT EXISTS) using Paxos; several round trips [S12].

Hot partition
One partition key receiving far more traffic than the rest (Lesson 7's hot key).

Write sharding
Spreading one hot key over several suffixed keys (Lesson 7's salting).

Access pattern
One way the app reads or writes data: the query, how often, how much, how fresh.

Polyglot persistence
More than one kind of datastore in one product, each for the pattern it serves best.

Source of truth / dual write
The store whose copy wins / the app writing one change to two stores itself.

CDC; graph DB; search engine
Change data capture (Lesson 10); two more families named only: graph and search.

INTERVIEW MODE: SAY IT LIKE THIS

- A partition key decides where data lives and a clustering or sort key orders it inside the partition. DynamoDB capacity is counted in read and write units, with per-partition limits.
- Embedding keeps related data in one document; referencing links documents. Polyglot persistence gives each access pattern the store that serves it best, with one source of truth per fact.

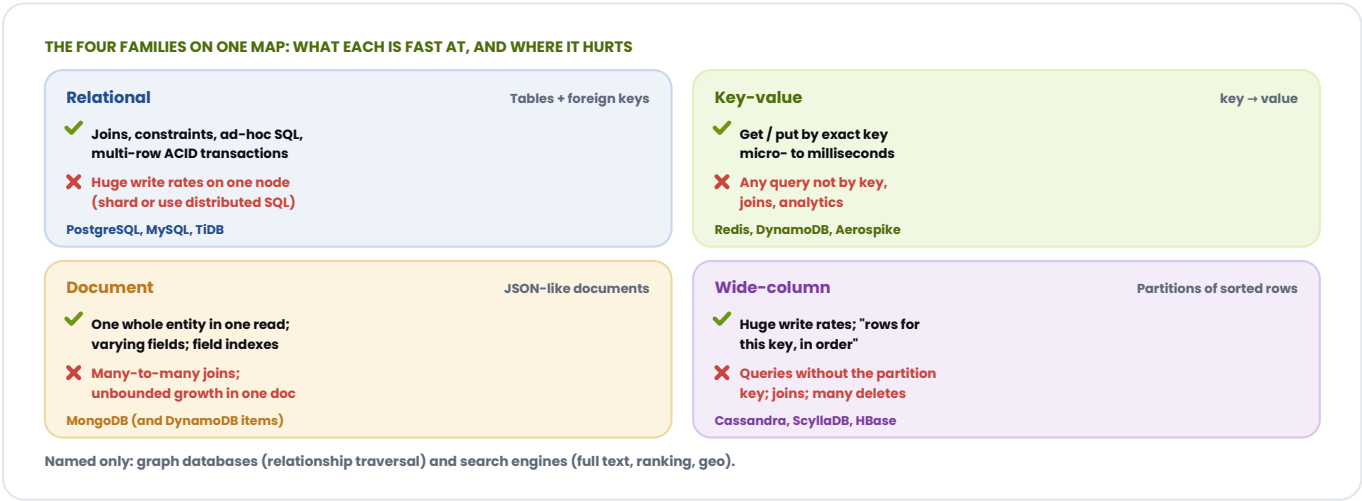
CONCEPT 1 · THE FOUR FAMILIES ON ONE MAP

"SQL or NoSQL?" is the wrong first question

Nine steps, bottom-up. The running examples are sho.rt (Lessons 1 to 7) and dabba.app, a fictional food-delivery app. Each concept has the problem it solves, how it works, a worked example with numbers, and what goes wrong. Numbers marked "assumption" are ours.

The problem it solves. Four families behave very differently. You need one map of what each is good and bad at before you can choose.

FAMILY	DATA MODEL	FAST AT	WEAK AT	EXAMPLES
Relational	Tables, rows, fixed schema, foreign keys	Joins, ad-hoc queries, constraints, multi-row ACID transactions	Very high write rates and sizes on one node without sharding (Lesson 7); schema changes on huge tables	PostgreSQL, MySQL; distributed SQL (TiDB)
Key-value	Key → value	Get and put by exact key, microseconds (memory) to single-digit ms (durable)	Any query not by key; joins; analytics	Redis, DynamoDB, Aerospike
Document	Collections of JSON-like documents	One whole entity in one operation; varying fields; field indexes	Many-to-many joins; unbounded growth in one document	MongoDB; DynamoDB items
Wide-column	Partitions of rows sorted by clustering key	Very high write throughput; "rows for this key, in order"; adding nodes	Queries not by partition key; joins; delete-heavy workloads	Cassandra, ScyllaDB, HBase, Bigtable



When someone says "we will use NoSQL", ask **which family**. Key-value, document and wide-column stores differ from each other as much as each differs from relational.

CONCEPT 1 · WHERE SHO.RT'S DATA FALLS

Shape is a hint, not a verdict

PAUSE AND PREDICT

sho.rt's links table is a perfect key-value shape. Must it move to a key-value store?

Answer: No. Lesson 7 showed it fits one PostgreSQL leader with replicas and Redis in front. A family "resembles" a pattern; only a limit you actually hit forces a move.

sho.rt'S DATA, SORTED BY ACCESS PATTERN (LESSONS 4 TO 7)

links	get by exact code, 7,716/s at peak	one key, one ~500 B value	Key-value
clicks	append 7,716/s; read one link by time range	many rows per key, by time	Wide-column
users, plans, invoices	joins; money must be exact	related tables + transactions	Relational
hot links (cache)	same small value read again and again	copy that may expire	Key-value

Shape is not destiny: links stay in PostgreSQL with Redis in front (Lesson 7). A family "resembles" the pattern; size decides if you need it.

Two more families appear in system design answers and are **named here only**: graph databases (relationship traversal: "people you may know") and search engines (full-text search and relevance: "biryani near me, open now").

WHAT GOES WRONG

Choosing by data shape alone ("it's JSON, so MongoDB"). The same JSON can be read by id, by owner, by date or by text search, and each of those reads prefers a different store.

Just for fun. Saying "we'll use NoSQL" in an interview is like saying "I'll travel by vehicle". Bus, bike or rocket? The interviewer is waiting.

TINY CHALLENGE

Which family does a "people you may know" feature resemble most? Answer: A graph store (named only): it walks friend-of-friend relationships.

MEMORY HOOK: FOUR SHAPES

Key → value. **Entity** → document. **Relations** → tables. **Stream per key** → wide-column.

INTERVIEW MODE: SAY IT LIKE THIS

- "There are four main families: relational for joins and transactions, key-value for exact-key lookups, document for whole entities with varying fields, and wide-column for append-heavy data read by key in order."

CONCEPT 2 · ACCESS PATTERNS DECIDE THE STORE

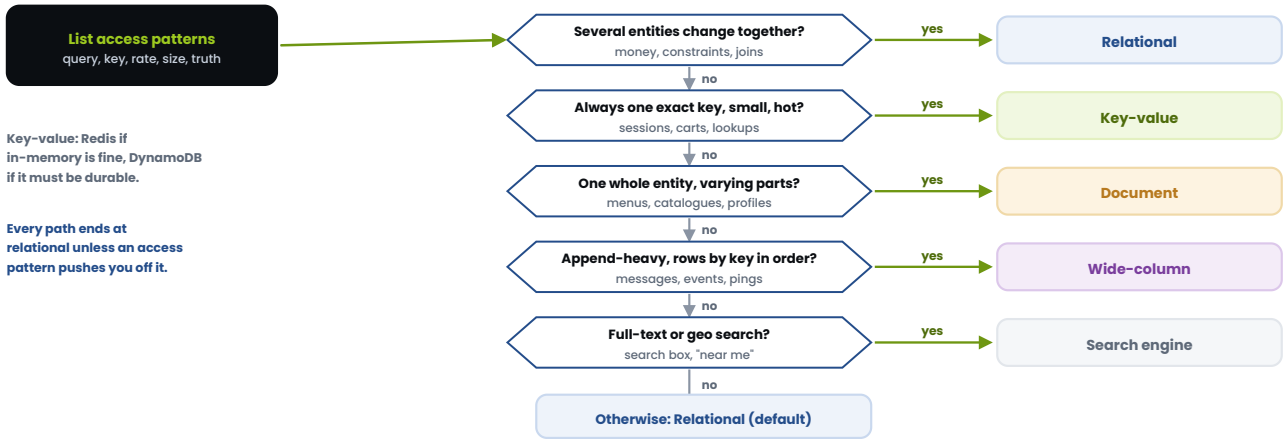
Write the list before you name a database

You need a repeatable way to pick a store that you can defend aloud, instead of naming the one you used last. Lesson 3's Concept 1 said "start from access patterns, not from tables". Here the same list picks the family.

COLUMN	QUESTION TO ANSWER
Query	What exactly is asked? ("latest 50 messages of channel X")
Key	An exact key, a key plus a range, or arbitrary fields?
Frequency	Per second at peak (Lesson 4)
Read/write mix	Mostly reads, mostly writes, or both?
Size and growth	Bytes per item, items per day, retention
Correctness	Must several changes commit together? Can a read be a second stale?

One line, filled in (sho.rt redirect, numbers from Lessons 4 to 7): **Query** long URL for a code. **Key** exact code. **Frequency** 7,716/s at peak, 772/s after the cache. **Mix** almost all reads, 77 creations/s. **Size** about 500 B, 6 billion items in five years. **Correctness** a second stale is fine, but a brand-new link must be visible at once (Lesson 7's hook).

SOURCE DIAGRAM 1, REDRAWN: FROM ACCESS PATTERNS TO A STORE



PAUSE AND PREDICT

Which of the six columns most often forces a relational database?

Answer: Correctness: several changes that must commit together (money, stock, seats), plus constraints like "email is unique".

CONCEPT 2 · THE DABBA.APP FEATURE LIST

One line per pattern, one store per line

Worked example (our assumptions: 2 million orders a day).

FEATURE	ACCESS PATTERN	STORE
Place an order and pay	Insert order, items, payment together; exact amounts	**Relational**
Cart while browsing	Get and put by session id; expires after 30 minutes	**Key-value** (Redis)
Restaurant menu	Read the whole menu by restaurant id; items and add-ons vary	**Document**
Delivery partner GPS	30,000 pings per second at dinner peak; read one partner's pings today in time order	**Wide-column**
"Suggested for you"	Precomputed list per (customer, area, meal slot); get by that key	**Key-value** (durable)
Search "paneer near me"	Full text, ranking, distance	**Search engine**

PAUSE AND PREDICT

A feature fits both relational and document equally well. Which do you pick?

Answer: The one your team already runs well, usually the relational database you already have. A second store has to earn its place.

WHAT GOES WRONG

Listing access patterns after choosing the store, to justify it. Write the list first; if two stores both fit, pick the one you already operate.

TINY CHALLENGE

A leaderboard: "top 10 scores", updated 5,000 times a second, a few seconds stale is fine. Which family, and which data type? Answer: Key-value: a Redis sorted set (ZADD, ZREVRANGE), with the scores stored durably elsewhere.

MEMORY HOOK: Q-K-F-M-S-C

Query, Key, Frequency, Mix, Size, Correctness. Six columns, then the tree.

INTERVIEW MODE: SAY IT LIKE THIS

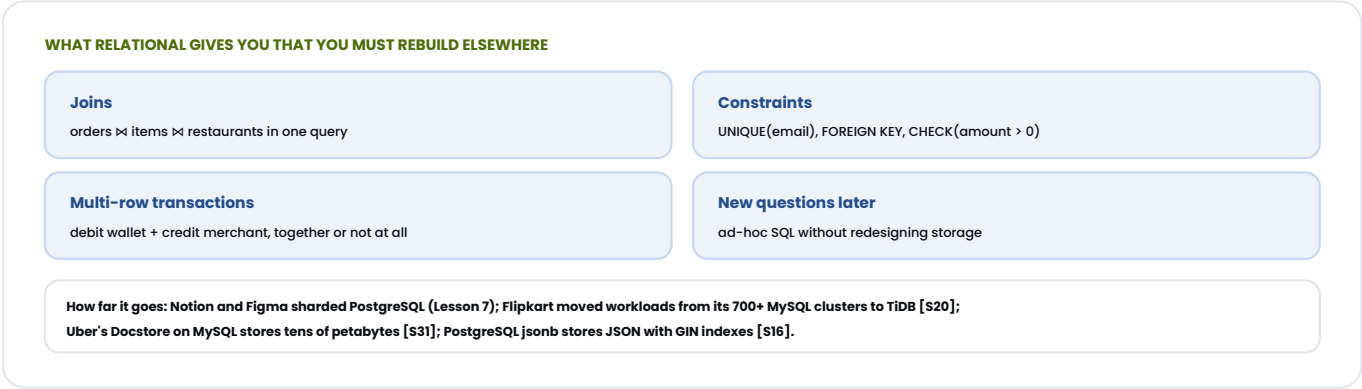
- "I list each access pattern with its key, peak rate, read/write mix, size and correctness need, then walk a short tree: transactions and joins go relational, exact-key lookups go key-value, whole entities go document, keyed streams go wide-column, search goes to a search engine."

CONCEPT 3 · RELATIONAL IS THE DEFAULT

How far SQL goes, and what NoSQL really offers

Students over-pick NoSQL. You need to know how far relational goes, and what the other stores really offer in 2026.

How it works. Joins, constraints and multi-row transactions mean you can add new queries later without redesigning storage (Lesson 3). It scales further than students think: Lesson 7 showed Notion and Figma sharding PostgreSQL. **Flipkart** ran over 700 MySQL clusters and moved many workloads to TiDB, a MySQL-compatible distributed SQL database, to stop sharding by hand while keeping SQL and ACID [S20] (a vendor-published case study). **Uber's Docstore**, built on MySQL, enforces schemas and supports transactions behind a document-style API [S21], and by 2024 stored tens of petabytes and served tens of millions of requests per second [S31]. PostgreSQL's jsonb type stores JSON in a binary form that is **"significantly faster to process"** than plain json and supports GIN indexes [S16], so "the data is JSON" alone is not a reason to leave Postgres.



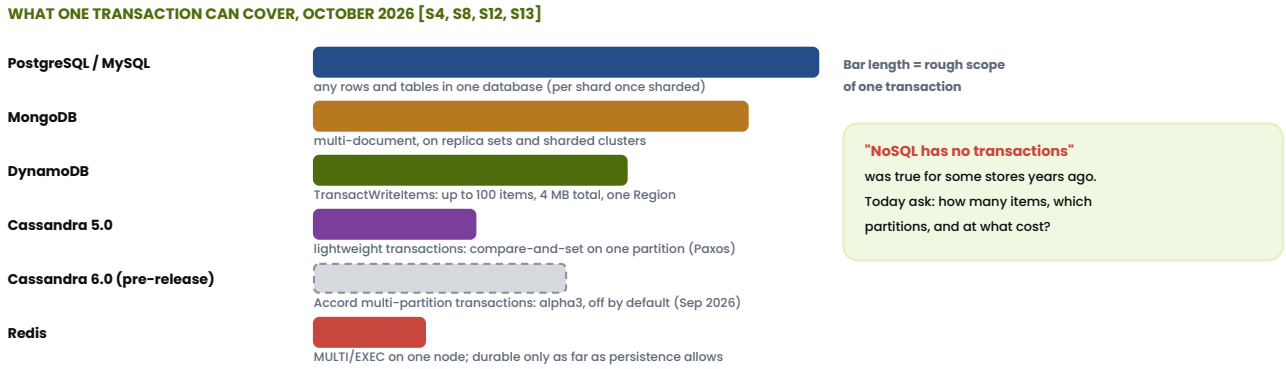
SIGNAL (LESSON 7'S FOUR LIMITS)	FIRST FIX, STILL RELATIONAL	WHEN ANOTHER STORE STARTS TO MAKE SENSE
Read throughput	Cache, read replicas	Exact-key reads at huge rates: key-value
Write throughput	Batching, sharding by a good key	Endless append-only streams: wide-column
Storage	Partitioning, archiving, sharding	Petabytes of keyed history: wide-column
Shape	jsonb columns with GIN indexes [S16]	Whole varying documents, rarely joined: document

```
-- flexible fields inside PostgreSQL, still indexed
CREATE TABLE restaurants (id bigint PRIMARY KEY, name text, attrs jsonb);
CREATE INDEX ON restaurants USING GIN (attrs);
SELECT id, name FROM restaurants WHERE attrs @> '{"pure_veg": true}';
```

CONCEPT 3 · TRANSACTIONS BY STORE, OCTOBER 2026

"NoSQL has no transactions" is out of date

STORE	WHAT ONE TRANSACTION CAN COVER
PostgreSQL / MySQL	Any number of rows and tables in one database (per shard once sharded, Lesson 7)
DynamoDB	TransactWriteItems: up to 100 distinct items, at most 4 MB in total, one account and Region [S4]
MongoDB	Multi-document transactions on replica sets and on sharded clusters ("distributed transactions") [S8]
Cassandra 5.0	Lightweight transactions: compare-and-set on one partition with Paxos and extra round trips [S12]. Multi-partition ACID (Accord) is in Cassandra 6.0, still pre-release (6.0-alpha3) in September 2026, with 5.0.9 the current stable release [S13]
Redis	MULTI/EXEC runs commands together on one node; durable only as far as its persistence setting (Concept 4)



Worked example (when "just Postgres" is right). A college canteen pre-order app: 3,000 orders a day, 3 KB per order with items and payment. 3,000 x 3 KB = **9 MB a day**, about **3.3 GB a year**. At 1 pm, say 40% of orders in one hour: 1,200 an hour, **0.33 per second**. One small PostgreSQL server with one replica is plenty for years.

WHAT GOES WRONG

"NoSQL scales, SQL does not." Both halves are wrong as written. Ask which limit you hit (Lesson 7's four limits) and whether replicas, sharding or distributed SQL cover it.

Just for fun. Choosing Cassandra for a canteen app with 3,000 orders a day is ordering a JCB to plant one tulsi plant.

TINY CHALLENGE

A transfer touches 3 DynamoDB items (two wallets and a ledger entry), each 2 KB. Within one transaction's limits? Answer: Yes: 3 items ≤ 100 and 6 KB ≤ 4 MB [S4].

MEMORY HOOK: RELATIONAL BY DEFAULT

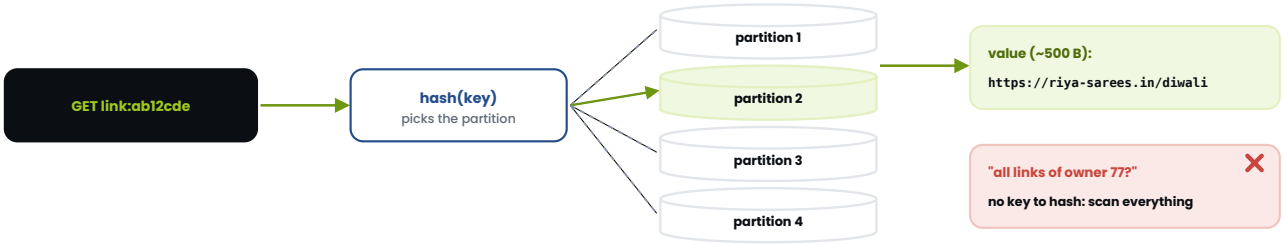
Start relational. Leave only when an access pattern and a number push you out.

CONCEPT 4 · KEY-VALUE STORES: REDIS AND DYNAMODB

Exact key in, one value out

Some data is always fetched by one exact key, very often: sessions, carts, rate-limit counters, feature flags, precomputed recommendations, sho.rt's code → URL. A store that does only that can do it faster and cheaper than anything else.

KEY-VALUE: HASH THE KEY, GO STRAIGHT TO ONE PARTITION, RETURN ONE VALUE



Use for: sessions, carts, rate limits, feature flags, precomputed lookups (Swiggy Suggests [S18]), sho.rt code → URL.

IF YOU NEED...	PICK	BECAUSE
Sub-millisecond reads; losing about a second of writes is acceptable, or the data can be rebuilt	**Redis**	In memory; AOF everysec may lose ~1 s [S14]
Leaderboards, rate limits, counters, short TTLs	**Redis**	Sorted sets, INCR, EXPIRE are built in [S15]
Data that must survive anything, with little to operate	**DynamoDB**	Durable, managed, replicated [S3]
Terabytes of keyed data (more than you want in RAM)	**DynamoDB**	Storage is not bounded by memory; partitions scale out
A second way to look items up	**DynamoDB** + GSI	Index on another key; reads eventually consistent [S5]

TWO VERY DIFFERENT KEY-VALUE STORES

Redis

Lives in	memory (persistence optional)
Latency	sub-millisecond
Durability	AOF everysec: may lose ~1 s [S14]
Types	strings, hashes, sorted sets, streams
Scales by	memory per node, cluster slots
Best as	cache, sessions, counters, leaderboards

DynamoDB

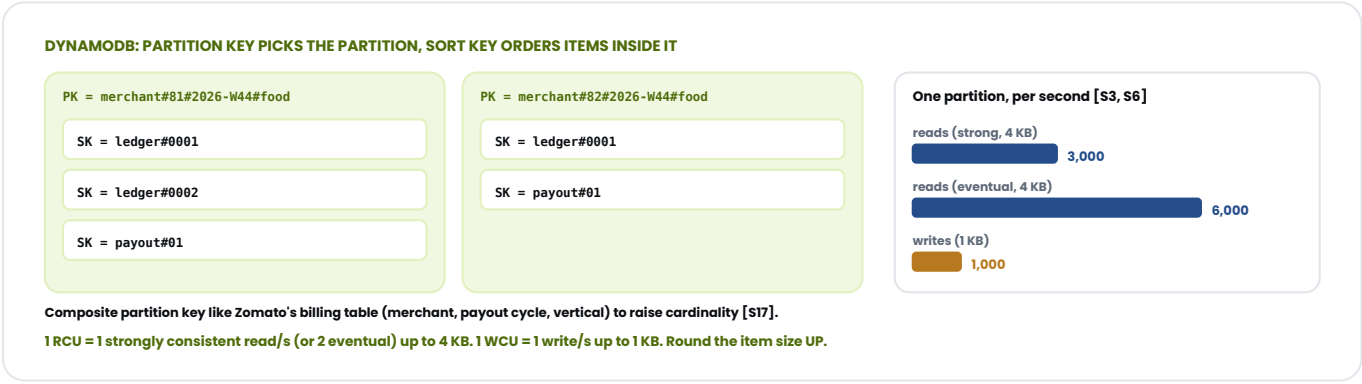
Lives in	durable storage, replicated by AWS
Latency	single-digit milliseconds [S23]
Durability	durable; strong or eventual reads
Types	items up to 400 KB [S4]
Scales by	partitions: 3,000 RCU / 1,000 WCU each [S3]
Best as	durable key lookups at any scale

Amazon SDE-2 (Jan 2026): Redis sorted set for the live leaderboard, but the winning bid lives in a durable store [S28].

CONCEPT 4 · WHO USES KEY-VALUE, FOR WHAT

Real use cases, sourced

- **Swiggy**'s "Swiggy Suggests" precomputes cart recommendations for each customer, geohash and meal slot in offline jobs and stores them "in a fast-access database such as DynamoDB"; the app reads them by that key [S18].
- **Swiggy** moved its centralised auth platform's data from an RDBMS to DynamoDB, "specifically designed for handling elasticity", with a live migration [S19].
- **Zomato** moved its billing platform from TiDB to DynamoDB: response time about 90% lower on average (about 75 ms), throughput from about 2,000 to 8,000 requests per minute, monthly cost about 50% lower [S17].
- **Amazon**, Prime Day 2026 (23 to 26 June): DynamoDB served over 59 trillion requests, peaking at **192 million requests per second** [S23].
- An **Amazon** SDE-2 candidate (January 2026) used Redis sorted sets for a live auction leaderboard and was pushed on why Redis cannot be the source of truth for the winning bid [S28]. Redis documents sorted sets for leaderboards and sliding-window rate limiters [S15].



USE CASE	KEY (OUR NAMING)	VALUE	EXPIRY
Session	session:{session_id}	user id, city, device	30 minutes
Cart	cart:{user_id}	items and quantities	24 hours
OTP attempts	otp:{phone}:{minute}	counter (INCR)	10 minutes
Idempotency key	idem:{request_id}	stored response	24 hours
Suggestions (Swiggy-style [S18])	sugg#{customer}#{geohash}#{slot}	ranked cart list	rebuilt nightly

```
# Redis: one key pattern per access pattern (key names are ours)
SET  session:9f2c  "{user:77, city:hyd}"  EX 1800   # session, 30-minute TTL
INCR otp:9876543210:202610291004              # attempts this minute
EXPIRE otp:9876543210:202610291004 600
ZADD auction:42:bids 1520 user:77               # leaderboard (sorted set)
ZREVRANGE auction:42:bids 0 9 WITHSCORES          # top 10
# DynamoDB: get by full key, or query one partition by a sort-key range
GetItem  key = { pk: "sugg#cust77#tdr1w#dinner" }
Query    pk = "merchant#81#2026-W44#food" AND begins_with(sk, "ledger#")
```

CONCEPT 4 · SHO.RT'S LINKS TABLE ON DYNAMODB

A per-partition limit is a hot-key limit in disguise

Worked example. Assumption: one link item is about 500 bytes (code, long URL, owner, created time).

QUANTITY	CALCULATION	RESULT
Read cost per lookup	500 B ≤ 4 KB: 1 RCU strong, 0.5 eventual [S6]	0.5 RCU (eventual is fine for redirects)
DB reads at peak (90% cache hit, Lesson 6)	772 x 0.5	**386 RCU**
Writes at peak	77 creations x 1 WCU (500 B ≤ 1 KB)	**77 WCU**
Storage after five years	6 billion x 500 B	about **3 TB** (Lesson 4's upper bound)
One viral link, no cache	20,000 reads per second on one key	**over** one partition's 6,000 eventual 4 KB reads per second [S3]

So DynamoDB could hold sho.rt's links easily, and the viral link still needs Lesson 6's cache.

PAUSE AND PREDICT

Would strongly consistent reads change the sho.rt bill much?

Answer: Reads would double from 386 to 772 RCU. For redirects a second-stale read is fine (new links are written through to Redis, Lesson 7), so eventual reads are the right default.

WHAT GOES WRONG

Using Redis as the only copy of data you cannot lose (orders, payments, the winning bid) with default settings. Decide its persistence (RDB, AOF), replicate it, or keep the truth in a durable store.

Just for fun. Redis without persistence is the friend who remembers everything you said at the party and nothing the next morning.

TINY CHALLENGE

An item is 12 KB. RCU for one strongly consistent read? For 300 such reads per second? Answer: 12 / 4 = 3 RCU; 300 x 3 = 900 RCU.

MEMORY HOOK: ROUND UP

Reads in **4 KB** steps, writes in **1 KB** steps, eventual reads **half price**. Per partition: **3,000 / 1,000**.

INTERVIEW MODE: SAY IT LIKE THIS

- "For exact-key lookups at high rate I use a key-value store: Redis when in-memory speed matters and a short loss is tolerable, DynamoDB when it must be durable."
- "I size DynamoDB in capacity units and check the hottest key against the per-partition limit of 3,000 read and 1,000 write units."

CONCEPT 5 · DOCUMENT STORES: MONGODB

Read the whole entity in one go

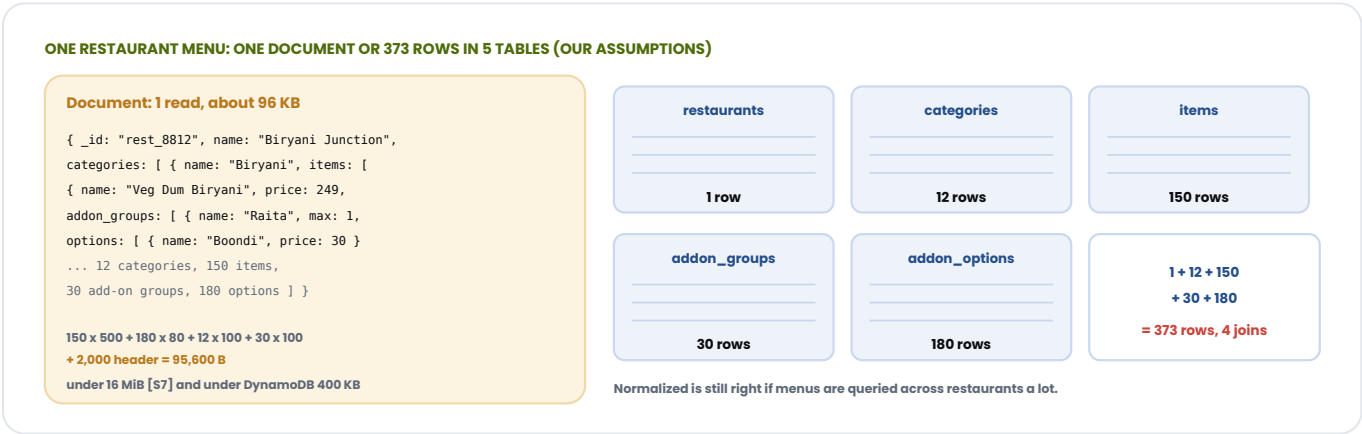
Some entities are read and written as a whole, with nested parts that differ from one entity to the next. Splitting them into many normalized tables makes every read a multi-table join and every new field a migration.

How it works. A MongoDB document is a JSON-like BSON object in a collection; you can index any field. Related data is either **embedded** or **referenced**. MongoDB's guidance: embed when entities have a "has-a" or "contains" relationship and are queried together; reference when embedded data would be duplicated and change often, when the child can exist on its own, or when you query the child on its own [S8]. Every document must be smaller than **16 mebibytes** [S7]. Schema validation can enforce required fields where you need it, and multi-document transactions exist for the cases that need them [S8].

PAUSE AND PREDICT

When is the normalized, five-table menu still the better design?

Answer: When the main queries cut **across** restaurants: "all items under Rs 150 tagged jain in Madhapur", price changes applied to many restaurants at once, or strict constraints between items and add-ons.



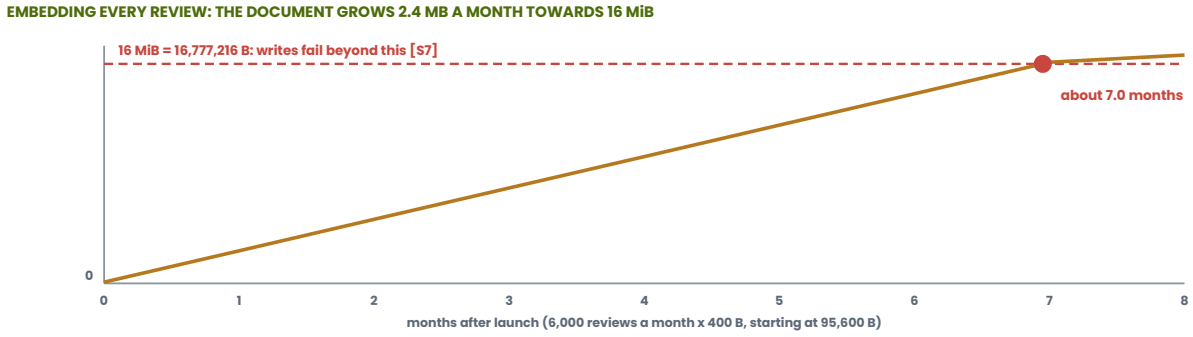
DESIGN	WHAT ONE "OPEN MENU" COSTS
Normalized (5 tables)	1 + 12 + 150 + 30 + 180 = **373 rows** from 5 tables, 4 joins
One embedded document	**1 read** of one document (about 96 KB)

```
{ "_id": "rest_8812", "name": "Biryani Junction", "area": "Madhapur",
  "categories": [ { "name": "Biryani", "items": [
      { "id": "it_1", "name": "Veg Dum Biryani", "price": 249,
        "addon_groups": [ { "name": "Raita", "max": 1,
            "options": [ { "name": "Boondi raita", "price": 30 } ] } ] } ] } ] }
```

CONCEPT 5 · THE DOCUMENT, AND THE UNBOUNDED ARRAY

Embed what is read together; reference what grows

Document size (our assumptions): 150 items x 500 B + 180 options x 80 B + 12 categories x 100 B + 30 groups x 100 B + 2,000 B header = 75,000 + 14,400 + 1,200 + 3,000 + 2,000 = **95,600 B**, about 96 KB. Far under 16 MiB, and under DynamoDB's 400 KB too.



WHAT GOES WRONG

Unbounded arrays. Someone embeds every review in the restaurant document. A popular restaurant gets 6,000 reviews a month at about 400 B each: 2.4 MB a month. Starting from 95,600 B it reaches 16 MiB in (16,777,216 - 95,600) / 2,400,000 ≈ **7.0 months**, after which every review write fails; long before that, every menu read drags megabytes of reviews.

Just for fun. Embedding all reviews in the menu is like stapling every customer complaint to the menu card. Very transparent. Very heavy.

TINY CHALLENGE

Embed or reference: a user's 3 saved delivery addresses? A user's full order history? Answer: Embed the addresses (few, read with the profile). Reference the orders (unbounded, queried alone).

MEMORY HOOK: FOLDER, NOT WAREHOUSE

A document is a **folder**, not a **warehouse**. Anything that grows forever gets its own collection.

INTERVIEW MODE: SAY IT LIKE THIS

- "I use a document store when one entity with varying nested parts is read and written as a whole, like a restaurant menu."
- "I embed data that the parent contains and that is read with it, and reference data that grows without limit, changes often or is queried on its own, keeping documents well under 16 MiB."

CONCEPT 6 · WIDE-COLUMN STORES: CASSANDRA

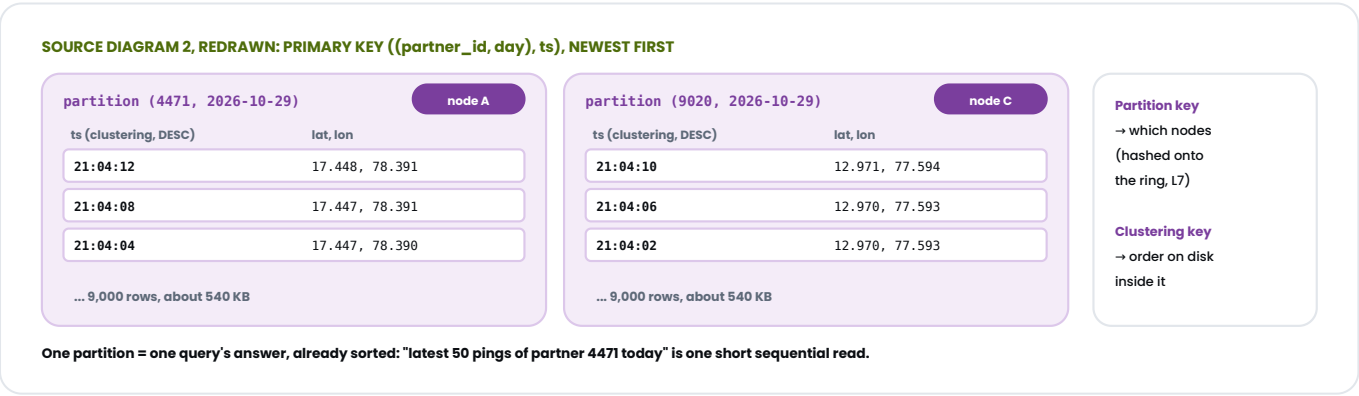
The doodhwala's notebook, at 30,000 lines a second

Some data arrives as a fast, never-ending stream of small rows (messages, GPS pings, order-status events, sensor readings), read as "the latest rows for this key". You want writes to stay cheap at any volume, and to add machines without re-sharding by hand.

How it works. In Cassandra, **"data modeling is query-driven"**: the queries determine the tables [S9]. The **partition key** decides which nodes hold the rows (a variant of consistent hashing, Lesson 7 [S9]); the **clustering columns** sort rows inside the partition on disk. You usually make one table per query and accept duplicated data. Discord's model is the classic: ((channel_id, bucket), message_id), with a time-sortable Snowflake message_id and buckets of about 10 days, so even the largest channels stay well under **100 MB per partition**, the size at which Discord saw garbage-collection pressure [S1].

```
CREATE TABLE partner_pings (
  partner_id bigint, day date, ts timestamp,
  lat double, lon double, speed float,
  PRIMARY KEY ((partner_id, day), ts)
) WITH CLUSTERING ORDER BY (ts DESC);

-- the one query this table exists for
SELECT ts, lat, lon FROM partner_pings
WHERE partner_id = 4471 AND day = '2026-10-29' LIMIT 50;
```



One table per query. If dispatch also needs "all pings for order 88123", you do not add an index to partner_pings; you write each ping a second time into `pings_by_order`, PRIMARY KEY ((order_id), ts). Writes are cheap here; duplicated data is the price of every read being one partition.

> DynamoDB uses the same idea under different names: **partition key** and **sort key**. If you can design a Cassandra table, you can design a DynamoDB table.

CONCEPT 6 · SIZING DELIVERY-PARTNER GPS

Bound every partition

Worked example (dabba.app, our assumptions): 200,000 partners, each online 10 hours a day, one ping every 4 seconds (900 per hour), 60 bytes per row.

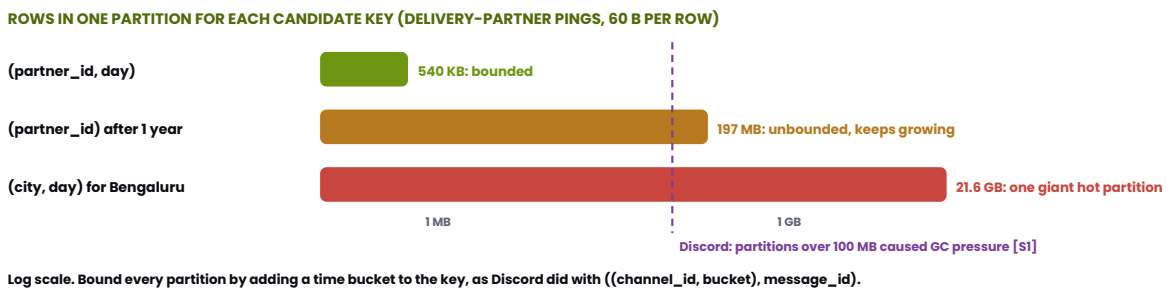
QUANTITY	CALCULATION	RESULT
Pings per partner per day	10 x 900	9,000
Rows per day	200,000 x 9,000	**1.8 billion**
Raw data per day	1.8 billion x 60 B	108 GB
With replication factor 3	108 x 3	324 GB a day
30 days kept, RF 3	324 GB x 30	about 9.72 TB
Peak writes (120,000 online at dinner)	120,000 / 4	**30,000 per second**

PAUSE AND PREDICT

Partition by (city, day). How big is Bengaluru's partition if it has 40,000 partners?

Answer: 40,000 x 9,000 = 360 million rows x 60 B = **21.6 GB** in one partition, written by everyone at once.

PARTITION KEY	ROWS IN ONE PARTITION	SIZE	VERDICT
(partner_id, day)	9,000	9,000 x 60 B = 540 KB	**Good**: bounded, 200,000 partitions a day
(partner_id) only	grows forever	540 KB x 365 ≈ 197 MB after a year	**Bad**: crosses 100 MB in under 7 months
(city, day)	360 million (Bengaluru)	21.6 GB	**Terrible**: one giant hot partition



Log scale. Bound every partition by adding a time bucket to the key, as Discord did with ((channel_id, bucket), message_id).

CONCEPT 6 · BUCKETS, AND THE NEW QUERY PROBLEM

Tell Cassandra the room number before it starts



WHAT GOES WRONG

Designing the table, then discovering a new query ("all partners who crossed this junction between 8 and 9 pm"). Cassandra cannot answer queries that do not start with the partition key without scanning. Cassandra 5.0's storage-attached indexes help with some filters [S11], but the usual answer is a second table shaped for the new query, or sending the data to an analytics or search system.

Just for fun. Cassandra is the friend who will carry your luggage up five floors without complaining, but only if you told him the room number before he started.

USE CASE	PARTITION KEY	CLUSTERING KEY	WHY IT FITS
Chat messages (Discord [S1])	(channel_id, bucket)	message_id DESC	Latest N per channel; append-heavy
Order status events	order_id	event_time	About 8 rows per order, read together
Smart meter readings	(meter_id, month)	reading_time	Endless appends; one meter's month at a time
Activity feed (precomputed)	(user_id, week)	post_time DESC	Read the newest items for one user

TINY CHALLENGE

30 days of one partner's pings in ONE partition (no day in the key). Size? Answer: 30 x 9,000 x 60 B = 16.2 MB. Fine for a month, but it never stops growing.

MEMORY HOOK: KEY = THE QUESTION

In wide-column, the **partition key is the question** and the **clustering key is the order of the answer**.

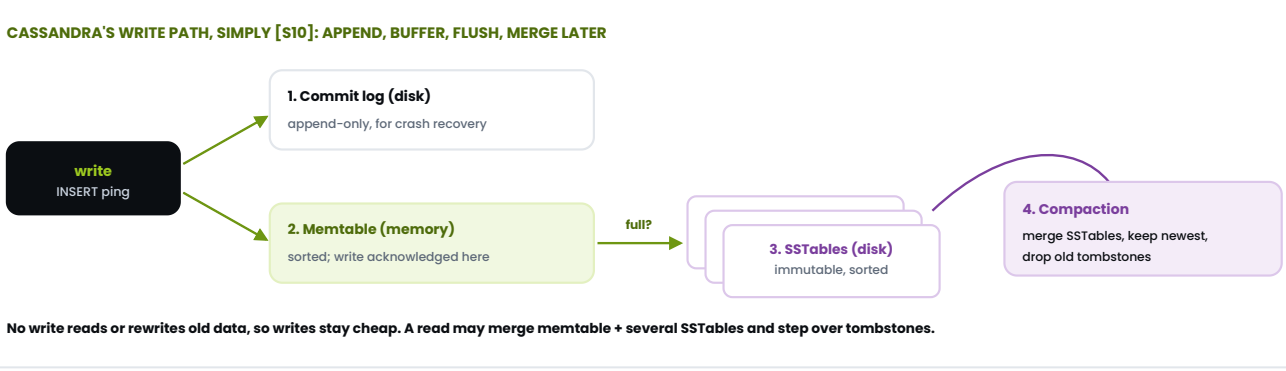
INTERVIEW MODE: SAY IT LIKE THIS

- "I use a wide-column store for append-heavy data read as rows for one key in order: messages, events, pings."
- "I design the table from the query: partition key for the node and the unit of the query, clustering key for the order, and a time bucket in the partition key so partitions stay bounded."

CONCEPT 7 · HOW A WIDE-COLUMN STORE WRITES

Append, buffer, flush, merge later

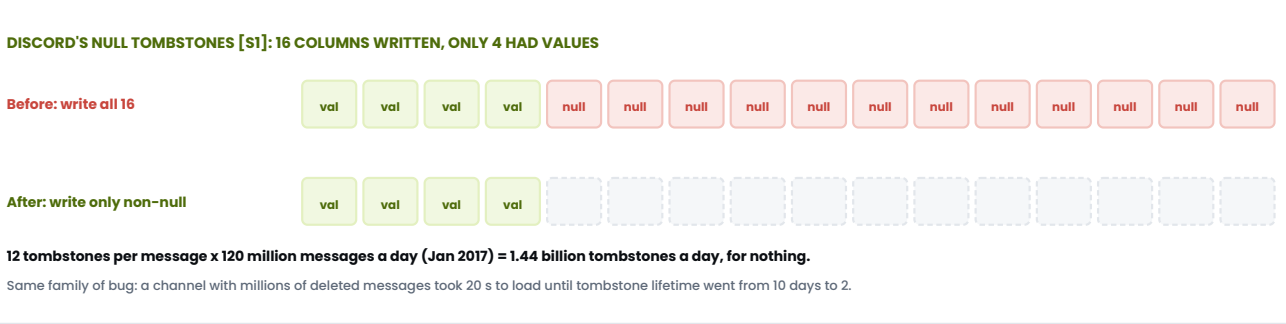
You need to explain, at a simple level, why Cassandra-style stores take writes so cheaply, and why deletes and nulls can make reads slow.



1. **Append** the write to the commit log on disk (sequential, for crash recovery) [S10].
2. **Write** it into the memtable, a sorted in-memory structure. The write is now acknowledged.
3. When the memtable is full, **flush** it to disk as an immutable, sorted SSTable.
4. In the background, **compaction** merges SSTables, keeps the newest version of each row and drops deleted data whose tombstones are old enough.

No write ever looks up or rewrites old data on disk, which is why writes stay fast. A read may check the memtable and several SSTables and merge them, which is why compaction matters. **A delete is also a write**: it writes a tombstone, and until compaction removes it, every read of that range steps over it.

WORKLOAD	EFFECT ON AN APPEND-AND-MERGE STORE
Append-only (messages, pings)	**Best case**: sequential writes, rows rarely change
Update-heavy (same row rewritten often)	**Older versions sit in several SSTables until compaction**
Delete-heavy or queue-like	**Tombstones pile up; reads step over them**
Writing nulls for empty columns	**Each null is a tombstone (Discord [S1])**



CONCEPT 7 · DISCORD'S TOMBSTONES

Writing null is deleting

Worked example. Discord found that writing a null value creates a tombstone just like a delete. An average message set only 4 of its 16 columns, so writing all 16 created about **12 unnecessary tombstones per message** [S1]. At the 120 million messages a day Discord reported in January 2017 [S1], that is 12 x 120 million = **1.44 billion tombstones a day** for nothing. The fix: write only non-null columns. Separately, a channel whose millions of messages were deleted through the API, leaving one, took **20 seconds** to load; Discord cut the tombstone lifetime from 10 days to 2 and tracked empty buckets [S1].

DISCORD'S PROBLEM [S1]	FIX
12 null tombstones per message	Write only non-null columns
20-second load after mass deletes	Tombstone lifetime 10 days → 2 (nightly repairs made it safe)
Queries scanning empty buckets	Track empty buckets; scan only the most recent

PAUSE AND PREDICT

Why is shortening the tombstone lifetime risky in general?

Answer: A replica that was down longer than the tombstone lifetime may never hear about the delete and can bring the row back. Discord could shorten it because repairs ran nightly [S1].

WHAT GOES WRONG

Using a wide-column table as a queue (insert a job, delete it when done). Every processed job leaves a tombstone, and "read the next job" steps over thousands of them. Use a real queue (Lesson 10).

Just for fun. A Cassandra delete is writing "this line is cancelled" in pen in the doodhwala's notebook. The page gets longer, not shorter.

TINY CHALLENGE

A table has 12 columns; messages set 4. How many tombstones per message if the app writes nulls? For 300 messages a second? Answer: 12 - 4 = 8 per message; 8 x 300 = 2,400 per second.

MEMORY HOOK: PEN, NOT PENCIL

Wide-column writes in **pen**: nothing is erased, only **crossed out** until compaction rewrites the page.

INTERVIEW MODE: SAY IT LIKE THIS

- "Cassandra writes append to a commit log and an in-memory memtable, which is flushed to immutable SSTables and compacted later, so writes never read old data."
- "Reads may merge several SSTables and skip tombstones, so I avoid writing nulls, avoid delete-heavy queue patterns, and keep compaction healthy."

CONCEPT 8 · KEYS, HOT PARTITIONS AND LIMITS

Lesson 7's hot key comes back

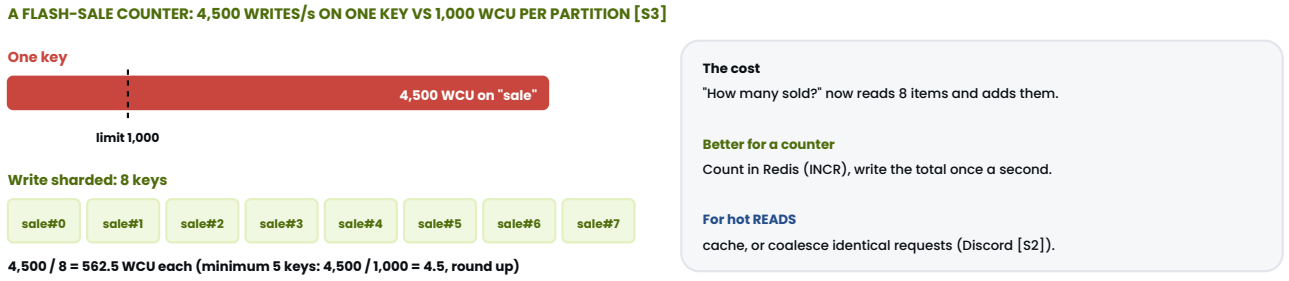
The hot key does not disappear when you pick a NoSQL store; it comes back as a hot partition, with hard per-partition limits.

- **DynamoDB:** every partition delivers at most 3,000 read units and 1,000 write units per second; AWS's own example is a 20 KB item, where one strongly consistent read costs 5 read units, so one item can be read at most 600 times a second before the partition limit [S3].
- **Cassandra/ScyllaDB:** one partition lives on a fixed set of replicas. Discord's hot partitions, "heavy traffic to one channel and time bucket", slowed a node and, through quorum reads and writes, other queries too. Discord added Rust "data services" that **coalesce** concurrent requests for the same row into one query, routing the same channel to the same instance [S2].
- **Good keys have many distinct values.** Zomato's billing table uses a composite partition key (merchant ID, payout cycle, business vertical) to "improve the cardinality", and its inverted index key carries a division-number suffix to avoid throttling [S17].

PAUSE AND PREDICT

A table has 50,000 WCU provisioned in total, but one key gets 4,500 writes a second. Throttled?

Answer: Yes, that key is. Total capacity is spread over partitions; one key lives in one partition with at most 1,000 WCU [S3].



Worked example (our assumptions). An "items sold" counter receives 4,500 updates per second during a flash sale; each item is under 1 KB, so 1 WCU per update.

QUANTITY	CALCULATION	RESULT
Write units needed	4,500 x 1	4,500 WCU per second
Minimum keys at 1,000 WCU each	4,500 / 1,000 = 4.5, round up	**5**
Chosen (headroom)	8 keys: sale#0 … sale#7	4,500 / 8 = **562.5** writes per second each
Read cost	Read all 8 and add	8 reads per "how many sold?"

CONCEPT 8 · BETTER KEYS

Table capacity scales; one key does not

LOW-CARDINALITY OR HOT KEY	BETTER KEY	WHY
status = "PLACED"	order_id (and a status index or table if needed)	Three statuses = three partitions for millions of orders
date	(user_id, date) or (device_id, date)	Today's date takes every write
merchant_id	(merchant_id, payout_cycle, vertical), as Zomato did [S17]	Big merchants dominate one partition
channel_id	(channel_id, bucket), as Discord did [S1]	Busy channels grow without limit
"sale" counter	sale#0 … sale#7, or aggregate in Redis	One key, 4,500 writes per second

This is Lesson 7's salting, inside DynamoDB. Better still for a counter: count in Redis and write the total once a second.

PAUSE AND PREDICT

Reads of the same hot key at 45,000 per second. Does write sharding help?

Answer: Not directly; it spreads writes. For hot reads: cache (Lesson 6), a CDN for public data, or request coalescing as Discord did [S2].

WHAT GOES WRONG

"DynamoDB scales automatically, so keys don't matter." Table capacity scales; a single key's partition does not.

Just for fun. Auto-scaling a table with one hot key is opening ten more ticket counters while everybody still queues at counter 1 because that is where the samosa is.

TINY CHALLENGE

7,000 writes per second on one key, items under 1 KB. Minimum shards? Answer: 7,000 / 1,000 = 7.

MEMORY HOOK: SPREAD, CACHE, COALESCE

Hot **writes**: spread the key. Hot **reads**: cache or coalesce. Hot **counters**: aggregate first.

INTERVIEW MODE: SAY IT LIKE THIS

- "I check every partition key for cardinality and for its peak rate against the per-partition limit."
- "For a hot key I use a composite key with more distinct values, write sharding with suffixes, caching or request coalescing for reads, and pre-aggregation for counters."

CONCEPT 9 · POLYGLOT PERSISTENCE IN ONE PRODUCT

Many stores, one truth per fact

A real product has many access patterns. One store rarely serves them all well; many stores bring their own costs. You need to draw the mix and defend every box.

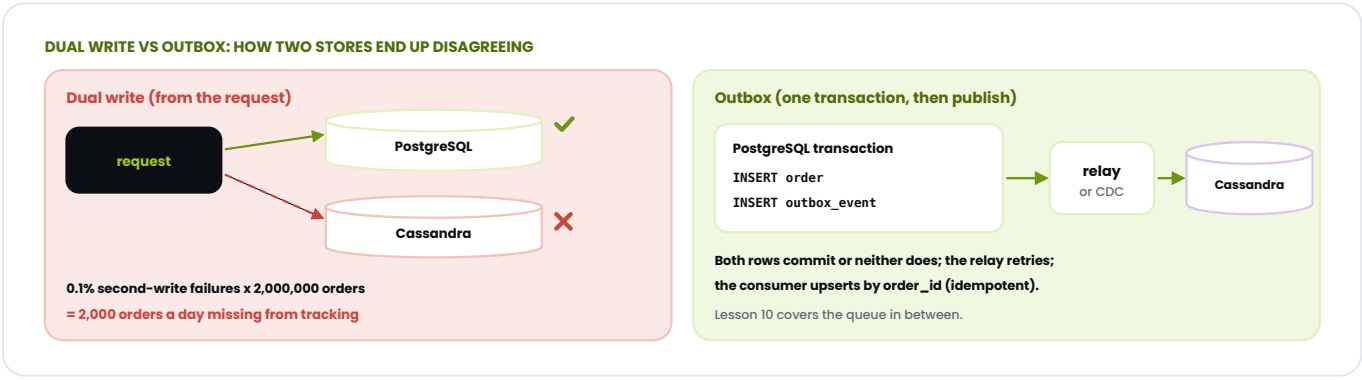
How it works. Each store gets one job and is the **source of truth** for specific facts. Changes flow from the source of truth to the others through events or change data capture, not through the application writing twice (**dual write**), because a dual write whose second half fails leaves the stores disagreeing. Each extra store is another thing to run, monitor, back up, secure and be paged for, so it must be earned by an access pattern the existing stores cannot serve.

PAUSE AND PREDICT

The order is saved in PostgreSQL, then the request crashes before writing Cassandra. Who notices?

Answer: Nobody, until a customer asks where the order is. That is why changes should flow from the source of truth, not from the request.

Interview evidence. Zomato interviewers asked a candidate why they used both PostgreSQL and MongoDB in one project [S25]; an Uber SDE-2 candidate had to justify SQL or NoSQL for the Uber Eats homepage's restaurants, dishes and orders [S27]. **Netflix** runs Cassandra for use cases "ranging from user sign-ups and storing viewing histories to supporting real-time analytics and live streaming", and built a key-value abstraction layer because service owners kept misusing datastores [S22]: the polyglot cost in one sentence.



EVERY EXTRA STORE ADDS	EXAMPLE QUESTION YOU MUST ANSWER
Operations and on-call	Who is paged when it is slow at 3 am?
Backups and restores	Can we restore it to the same moment as PostgreSQL?
Security and access	Which services may read it, and is it encrypted?
Skills	Who on the team has tuned it before?
Consistency with the others	Which store wins if they disagree, and how do changes flow?

CONCEPT 9 · THE DUAL-WRITE GAP, IN NUMBERS

Earn each store with a number

Worked example (our assumptions). dabba.app writes each order to PostgreSQL, then writes an "order placed" event to the Cassandra tracking table from the same request. If the second write fails 0.1% of the time, 2,000,000 x 0.001 = **2,000 orders a day** exist in PostgreSQL but never appear in tracking. Fix: write the order and an **outbox row in one PostgreSQL transaction**, and let a relay or CDC publish it (Lesson 10).

WHAT GOES WRONG

Six stores for a startup with 2,000 orders a day, because the architecture diagram looked like a unicorn's. Earn each store with a number.

Just for fun. Polyglot persistence is like speaking five languages. Impressive at a conference. Exhausting if you must also translate every conversation between them at 3 am.

TINY CHALLENGE

Same app, 0.02% failure on the second write. Orders missing from tracking per 30 days? Answer: 2,000,000 x 0.0002 = 400 a day; x 30 = 12,000.

MEMORY HOOK: ONE TRUTH, MANY COPIES

One source of truth per fact. **Many** copies are fine if they flow **from** it.

INTERVIEW MODE: SAY IT LIKE THIS

- "I list the access patterns first: query, key, frequency, read/write mix, size and correctness. Relational is my default for joins, constraints and money."
- "Key-value for exact-key lookups at high rate, documents for whole entities with varying parts, wide-column for append-heavy keyed streams."
- "I check partition keys for cardinality and limits, bound partition sizes, and keep one source of truth per fact, with changes flowing by outbox or CDC rather than dual writes."

CONCEPT	ONE LINE	ONE NUMBER
1 · Four families	Key, entity, relations, stream per key	sho.rt: 4 data sets, 3 families
2 · Access patterns	Q-K-F-M-S-C, then the tree	6 dabba.app features, 5 store types
3 · Relational default	Leave only when a number pushes you	Canteen app: 9 MB a day
4 · Key-value	Redis in memory, DynamoDB durable	sho.rt links: 386 RCU, 77 WCU
5 · Document	Embed what is read together	Menu 95,600 B; reviews hit 16 MiB in 7.0 months
6 · Wide-column	Partition = question, clustering = order	(partner_id, day): 540 KB vs 21.6 GB
7 · Write path	Append, buffer, flush, merge	Discord: 1.44 billion null tombstones a day
8 · Hot partitions	Spread, cache, coalesce	4,500 WCU → at least 5 keys
9 · Polyglot	One truth per fact, no dual writes	0.1% gap = 2,000 orders a day

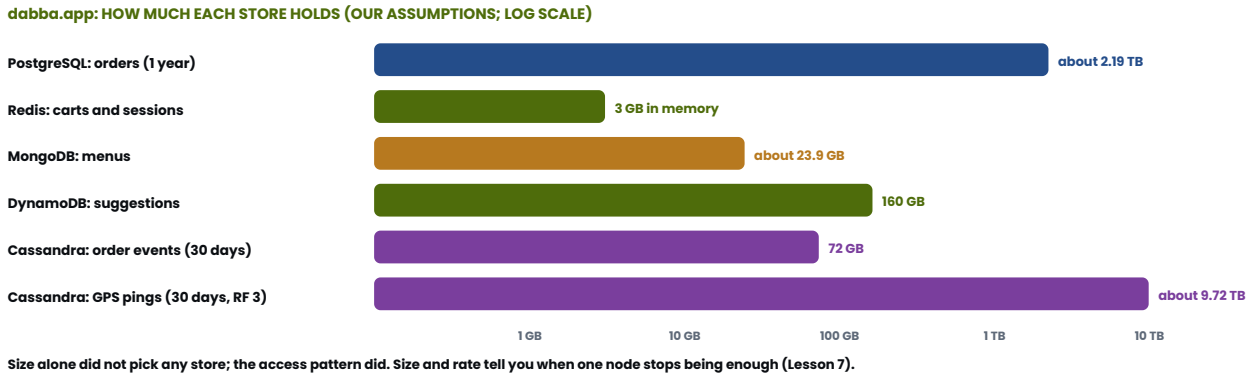
ONE WHITEBOARD, EVERY NUMBER

The data layer for dabba.app

Requirements (our assumptions). A fictional food-delivery app in 40 cities: 2 million orders a day, 12% in the busiest hour; 250,000 restaurants; 20 million customers; 200,000 delivery partners online 10 hours a day; menus opened 60 million times a day; the app opened 30 million times a day; 1.5 million concurrent sessions at peak.

Step 1: list the access patterns and size them.

DATA	ACCESS PATTERN	RATE	SIZE	STORE
Orders, items, payments	Insert together; read by order and by customer; refunds, reports	busiest hour: 2,000,000 x 0.12 / 3,600 ≈ 66.7 orders/s	3 KB per order: 6 GB a day, about 2.19 TB a year	**PostgreSQL** (truth for orders and money)
Carts and sessions	Get/put by session id, TTL 30 min	every tap	1.5 million x 2 KB = 3 GB	**Redis**
Menus	Read whole menu by restaurant id	60 million / 86,400 ≈ 694/s average, about 2,083/s at 3x	250,000 x 95,600 B ≈ 23.9 GB	**MongoDB** (+ Redis for hot menus)
Order status events	Append about 8 per order; read one order's events in order	16 million a day	150 B each: 2.4 GB a day	**Cassandra**: partition order_id, clustering event_time
Partner GPS pings	Append; read one partner's day	30,000/s at peak	108 GB a day raw	**Cassandra**: (partner_id, day), ts
"Suggested for you"	Precomputed nightly; get by (customer, geohash, meal slot)	30 million / 86,400 ≈ 347/s average	80 million items x 2 KB = 160 GB	**DynamoDB** (as Swiggy describes [S18])
Restaurant search	Text + distance + open now + ranking	every search	built from MongoDB and PostgreSQL	**Search engine** (named only)



Size alone did not pick any store; the access pattern did. Size and rate tell you when one node stops being enough (Lesson 7).

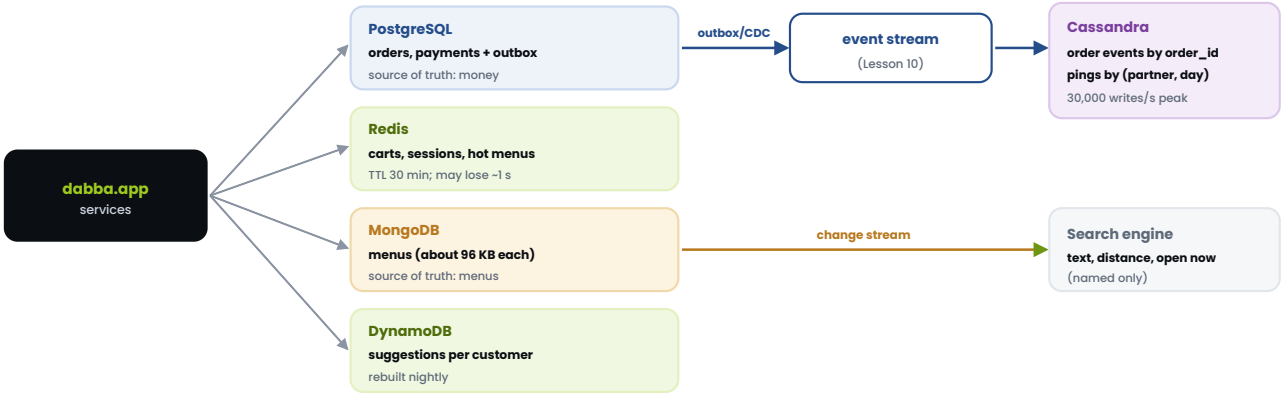
STEPS 2 TO 5

Truth, sync, the worst day, day one

Step 2: name the source of truth for each fact. Order and payment state: PostgreSQL. Menu content: MongoDB. Location history: Cassandra. Recommendations: recomputable, so DynamoDB holds a rebuildable copy. Carts: Redis, acceptable to lose a few seconds of cart edits.

Step 3: connect them without dual writes. Orders commit with an outbox row in the same PostgreSQL transaction; a relay publishes "placed / picked up / delivered" events that write the Cassandra status table and update caches. Menu changes in MongoDB feed the search index through its change stream. Nightly jobs write DynamoDB.

SOURCE DIAGRAM 3, REDRAWN: dabba.app'S DATA LAYER, ONE JOB PER STORE



Arrows from each source of truth replace dual writes. At 2,000 orders a day, PostgreSQL (with jsonb and a partitioned events table) + Redis is enough.

Step 4: check the keys against the worst day. Diwali dinner: one viral restaurant's menu is a hot document, so cache it in Redis and locally (Lesson 6). A cloud-kitchen hub with 2,000 orders an hour: its status events spread by order_id, so no hot partition. Partner pings spread across 200,000 (partner_id, day) partitions.

Step 5: decide what you would not do on day one. At 2,000 orders a day instead of 2 million, everything fits in PostgreSQL (orders, a jsonb menu column, a partitioned events table) plus Redis. Add each other store only when its access pattern and number demand it.

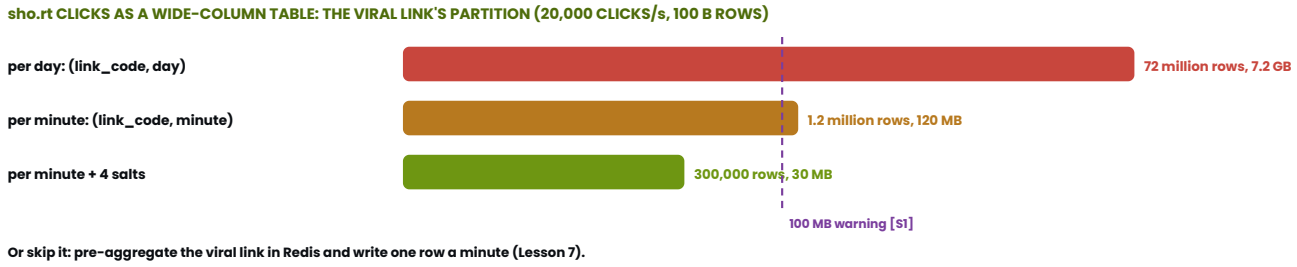
DATA	DAY ONE (2,000 ORDERS A DAY)	AT 2 MILLION ORDERS A DAY
Orders, payments	PostgreSQL	PostgreSQL, sharded by customer later (Lesson 7)
Carts, sessions	Redis	Redis cluster
Menus	PostgreSQL jsonb column + Redis cache	MongoDB + Redis cache
Status events, pings	Partitioned PostgreSQL table, kept 30 days	Cassandra
Suggestions	A table refreshed nightly	DynamoDB
Search	PostgreSQL full-text + a geo index	Search engine

WHERE SHO.RT LANDS

links stay put; clicks fit a wide-column table

links stay in PostgreSQL with Redis in front (Lesson 7's decision); DynamoDB would also fit, at about 386 RCU and 77 WCU at peak. **clicks** are a natural wide-column table: partition (link_code, day), clustering clicked_at. One caution from Lesson 7's viral link: 20,000 clicks per second for an hour is 72 million rows, 7.2 GB, in one (link_code, day) partition.

```
CREATE TABLE clicks (
  link_code text, day date, clicked_at timestamp, click_id timeuuid,
  country text, device text, referrer text,
  PRIMARY KEY ((link_code, day), clicked_at, click_id)
) WITH CLUSTERING ORDER BY (clicked_at DESC, click_id ASC);
-- click_id keeps two clicks in the same instant apart
```



BUCKET	ROWS IN THE VIRAL LINK'S PARTITION	SIZE AT 100 B PER ROW
Day	20,000 x 3,600 = 72 million (one hour of virality)	7.2 GB
Minute	20,000 x 60 = 1.2 million	120 MB
Minute + 4 salts	1.2 million / 4 = 300,000	30 MB

INTERVIEW MODE: SAY IT LIKE THIS

- "For a food-delivery app I keep orders and payments in a relational database as the source of truth, carts and sessions in Redis, menus as documents, order-status events and partner locations in a wide-column store keyed by order and by partner-day, and precomputed recommendations in a durable key-value store."
- "Restaurant search goes to a search engine fed from the sources of truth. Changes flow through an outbox or CDC, not dual writes, and at small scale I would start with PostgreSQL and Redis only."

THINGS THAT SOUND RIGHT AND ARE NOT

Common misconceptions

THEY SAY	THE TRUTH
"NoSQL is faster than SQL."	Speed depends on the access pattern. A key lookup is fast in any of them; a five-entity join is slow or impossible in a key-value store.
"NoSQL databases have no transactions."	DynamoDB transactions cover up to 100 items and 4 MB [S4]; MongoDB has multi-document transactions [S8]; Cassandra has single-partition LWT, with Accord arriving in 6.0 [S12, S13].
"SQL databases cannot scale horizontally."	Sharded PostgreSQL and MySQL run at very large scale (Lesson 7); distributed SQL such as TiDB scales out with SQL and ACID [S20].
"Schemaless means no schema."	The schema moves into the application; MongoDB supports validation [S8], and Uber's Docstore deliberately enforces schemas [S21].
"DynamoDB scales automatically, so the key does not matter."	Each partition is limited to 3,000 RCU and 1,000 WCU [S3]; one hot key is still one partition.
"Cassandra is good at everything big."	It is good at writes and reads by partition key. Queries without it, joins and delete-heavy workloads hurt [S1, S9].
"Redis is just a cache."	It is also a primary store for data that tolerates its persistence model; sorted sets and streams solve real problems [S14, S15].
"Embed everything in one document for speed."	Unbounded arrays hit the 16 MiB limit and make every read heavy [S7, S8].
"Polyglot persistence is a best practice."	It is a cost you pay for access patterns that need it, not a goal [S22].

Just for fun. "We moved to NoSQL and everything got faster" usually means "we finally added the index we needed". The database gets the credit; the index did the work.

INTERVIEW MODE: SAY IT LIKE THIS

- "Speed and scale depend on access patterns, not on the SQL or NoSQL label. Most NoSQL stores have some transactions now, SQL scales with sharding and distributed SQL, and per-partition limits and partition sizes still need design."

TEAR OUT THIS PAGE

One-page cheat sheet

WHAT THEY ASK	THE ANSWER	THE TRAP
SQL or NoSQL?	Neither until I list the access patterns; relational by default	Answering before asking about queries
When key-value?	Exact-key reads and writes at high rate: sessions, carts, rate limits, precomputed lookups	Needing any query not by key
Redis or DynamoDB?	Redis: in memory, sub-ms, persistence optional. DynamoDB: durable, single-digit ms, per-partition limits [S3, S14]	Redis as the only copy of money
When document?	One entity read as a whole with varying nested parts: menus, catalogues, profiles	Unbounded arrays; many-to-many joins
Embed or reference?	Embed "contains" data read together; reference data that grows, changes often or stands alone [S8]	Hitting 16 MiB [S7]
When wide-column?	Append-heavy, read as rows for a key in order: messages, events, pings	Queries without the partition key
Partition vs clustering key?	Partition: which node, and the unit of a query. Clustering: order inside the partition	A timestamp alone as partition key
Why are Cassandra writes fast?	Commit log append + memtable; flush to SSTables; compaction later [S10]	Forgetting tombstones
DynamoDB capacity?	1 RCU = 1 strong (2 eventual) reads/s up to 4 KB; 1 WCU = 1 write/s up to 1 KB [S6]	Not rounding item size up
Hot partition?	High-cardinality composite keys; write sharding; cache or coalesce reads [S2, S17]	"Auto-scaling fixes it"
Transactions in NoSQL?	DynamoDB 100 items / 4 MB; MongoDB multi-document; Cassandra LWT, Accord in 6.0 [S4, S8, S12, S13]	"NoSQL has no transactions"
Polyglot?	One job and one source of truth per store; sync with outbox or CDC	Dual writes; six stores at small scale

3,000 / 1,000
DynamoDB RCU / WCU per partition [S3]

400 KB
DynamoDB item maximum [S4]

100 / 4 MB
items / bytes per DynamoDB transaction [S4]

16 MiB
MongoDB document maximum [S7]

100 MB
Discord's partition warning size [S1]

~1 s
Redis AOF everysec worst-case loss [S14]

INTERVIEW MODE: SAY IT LIKE THIS

- "I answer the store question by listing access patterns, default to relational, and pick key-value, document or wide-column only when a pattern needs it, then check keys, partition limits and sizes, and how the stores stay in sync."

WHAT REAL TEAMS ACTUALLY DID

Where this shows up in real systems

SYSTEM	WHAT IT DID	CONCEPT
Discord (2017)	MongoDB replica set → Cassandra when data and index outgrew RAM at about 100 million messages; ((channel_id, bucket), message_id), 10-day buckets, partitions under 100 MB; 12 nodes, RF 3; null tombstones fixed [S1]	Hook, 6, 7
Discord (2023)	177 Cassandra nodes → 72 ScyllaDB nodes; hot partitions, GC pauses, compaction backlog; p99 fetch 40–125 ms → 15 ms; Rust data services with request coalescing; migration up to 3.2 million messages/s [S2]	6, 8
Zomato (2023)	Billing: TiDB → DynamoDB; single-table adjacency list, composite partition key, GSI with division suffix; about 90% lower response time, 4x throughput, about 50% lower cost [S17]	4, 8
Swiggy (2023, 2024)	Swiggy Suggests served from DynamoDB per customer-geohash-meal slot [S18]; auth platform RDBMS → DynamoDB with a live migration [S19]	4
Flipkart (2023, vendor-published)	Over 700 MySQL clusters; workloads moved to TiDB to stop application-level sharding while keeping SQL and ACID [S20]	3
Uber (2016 to 2024)	Schemaless on MySQL → Docstore with schemas and transactions [S21]; tens of PBs, tens of millions of requests/s by 2024 [S31]	3
Netflix (2024)	Cassandra for sign-ups, viewing history, real-time analytics, live streaming; a key-value abstraction layer to stop datastore misuse [S22]	6, 9
Amazon Prime Day 2026	DynamoDB: over 59 trillion requests in four days, peak 192 million per second, single-digit ms [S23]	4

Just for fun. Discord's blog title went from "billions" in 2017 to "trillions" in 2023. At this rate the 2029 post needs a new word, and possibly a new number system.

NUMBERS WORTH REMEMBERING

DynamoDB per partition: **3,000 RCU / 1,000 WCU**; item **400 KB**; transaction **100 items / 4 MB**. MongoDB document: **16 MiB**. Discord: partitions under **100 MB**, buckets of **10 days**, **177 → 72** nodes. Redis AOF everysec: lose at most about **1 s**.

INTERVIEW MODE: SAY IT LIKE THIS

- "Discord moved messages to Cassandra and later ScyllaDB because the access pattern was latest messages in a channel at huge volume. Zomato and Swiggy use DynamoDB for key-shaped, elastic workloads; Flipkart and Uber stayed with SQL engines and scaled them out."

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 1 to 3

Six questions are Source-confirmed (Zomato x2, Swiggy, Uber, Amazon x2), one is Candidate-reported (Infosys, aggregator), the rest are Pattern-only. Follow-up lines are written for this PDF from the lesson's own content.

Q1 What is Postgres? Why didn't you use MongoDB? When should you use Postgres? Does Postgres satisfy ACID? SOURCE-CONFIRMED

Zomato (Eternal), SDE-1, online interview round, July 2025 [S24].

Model answer. Tie it to your project's access patterns. "My data had relations (users, projects, documents, chats) and I needed transactions when creating a project with its first documents, so a relational database fit. Postgres is fully ACID. Flexible metadata sits in a jsonb column with a GIN index, so I did not need a document store. I would pick MongoDB if each record were one self-contained document with varying fields, read as a whole, with few joins."

> **"Is CAP relevant to Postgres?"** "For a single node, partitions are not the issue; once I replicate across nodes, I choose what happens during a partition. Lesson 9 covers it properly."

Weak: "MongoDB is for big data, Postgres is for small data."

Strong: The access pattern, the transaction need, and when the other choice would win.

Q2 Why PostgreSQL? Why use both PostgreSQL and MongoDB in the same project? SOURCE-CONFIRMED

Zomato, SDE Intern (6 months), Round 1, April 2026 [S25].

Model answer. "Postgres held users and payments, which need joins and transactions. MongoDB held per-course content documents whose structure varied and were read whole. Each fact has one home. The cost is two systems to run and keep in sync; at small scale I would use a jsonb column instead."

> **"How do the two stay consistent?"** "Each fact lives in one store only; if one store must react to the other, I publish an event from the source of truth instead of writing both from the request."

Weak: "Mongo is faster for reads."

Strong: One access pattern per store, one source of truth per fact, and the cost.

Q3 Discuss the differences, use cases, advantages and disadvantages of SQL and NoSQL databases. SOURCE-CONFIRMED

Swiggy, SDE II Backend, DSA + backend round, interviewed November 2024 (posted June 2025) [S26].

Model answer. Relational: fixed schema, joins, constraints, multi-row transactions, ad-hoc queries; scales with replicas and sharding. NoSQL is four families: key-value (exact-key lookups), document (whole entities with varying fields), wide-column (append-heavy, rows by key in order), graph. Each is fast at its own access pattern and weak outside it. Most have transactions now, with limits. Choose by access pattern and correctness, not by label.

> **"Give one example of each in a food app."** "Orders relational, cart in Redis, menu as a document, delivery-partner pings in Cassandra."

Weak: "SQL is structured, NoSQL is unstructured."

Strong: Four families, each with its access pattern, plus transactions in 2026.

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 4 to 6

Q4 Design the Uber Eats homepage: schema for restaurants, dishes and orders, and your choice of database (SQL vs NoSQL) with justification. SOURCE-CONFIRMED

Uber, SDE-2 (L4) Bengaluru, HLD round, September 2025 [S27].

Model answer. Orders and payments relational. Restaurant and dish catalogue as documents (or relational with a cache), because the homepage reads a restaurant card as a whole. "Restaurants near me" through a geo/search index. The homepage served mostly from caches. Metrics like "most ordered dish" from an event stream into counters, not by scanning orders.

> **"Most ordered dish across all restaurants?"** "Count per dish from order events into a counter table; if orders are sharded by restaurant, add per dish first, then rank (Lesson 7, Problem 5)."

Weak: "One MongoDB for everything."

Strong: A store per access pattern and a cache for the read-heavy homepage.

Q5 Could you build an inventory service on NoSQL? (Same loop: a Redis sorted set for a live auction leaderboard; why Redis cannot be the source of truth for the winning bid.) SOURCE-CONFIRMED

Amazon, SDE-2, January 2026 [S28].

Model answer. Yes if the access pattern is "stock for this SKU at this store" and decrements are conditional: DynamoDB conditional updates or transactions (up to 100 items) prevent overselling [S4]. Cross-item reporting goes to a warehouse. The leaderboard lives in Redis for speed, but bids are stored durably first and Redis is rebuilt from them.

> **"What if Redis restarts mid-auction?"** "Rebuild the sorted set from the durable bids table; with AOF everysec at most about a second of updates is lost from Redis anyway [S14]."

Weak: "NoSQL can't do inventory because no transactions."

Strong: Conditional writes, transaction limits, and where the truth lives.

Q6 Design a blogging platform: which database would you choose? SOURCE-CONFIRMED

Amazon, SDE II (L5), system design round, May 2025 [S29].

Model answer. Posts and users relational (or posts as documents, read whole). Comments in their own table or collection keyed by post and time, never an unbounded array. View counters in Redis. Search through a search engine. Feeds precomputed per user if read-heavy.

> **"A post gets 200,000 comments."** "Comments are their own collection, indexed by (post_id, created_at), paged; the post document never grows."

Weak: "NoSQL because blogs are unstructured."

Strong: Per-feature stores, the comment trap, counters and search.

Notice the pattern in Q4 to Q6: nobody asked "define NoSQL". Each interviewer gave a **product** and waited for you to choose stores **per feature**, with reasons.

Just for fun. Saying "I'll use MongoDB because it's web scale" in 2026 is like saying "I'll use a Nokia because it's unbreakable". True-ish. Not the point.

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 7 to 10

Q7 Why did you choose MongoDB over MySQL for your project? CANDIDATE-REPORTED

Infosys, System Engineer, aggregator listing, interview January 2026 [S30].

Model answer. Give the access pattern, not "flexible schema" alone: "Each quiz attempt is one document with a varying list of answers, read and written as a whole; no joins on the hot path." Then say what you gave up (joins, some constraints) and how you handled it (validation rules, a separate collection for reports).

> **"What would make you switch back?"** "Reports that join attempts with students and colleges every day; a relational store or an analytics copy would serve them better."

Weak: "MongoDB is modern."

Strong: The access pattern, the trade-off, and the switch condition.

Q8 Explain partition key vs clustering key in Cassandra, and design a table for chat messages. PATTERN-ONLY

Model answer. The partition key decides the nodes and is the unit of a query; the clustering key orders rows inside the partition. Messages: ((channel_id, bucket), message_id) clustered newest first, bucketed by time so partitions stay bounded, as Discord did [S1].

> **"Why the bucket?"** "Without it a busy channel's partition grows forever; Discord saw GC pressure above 100 MB [S1]."

Weak: "Partition by timestamp."

Strong: Partition = question, clustering = order, bucket = bound.

Q9 Why are writes fast in Cassandra, and what makes reads slow? PATTERN-ONLY

Model answer. A write appends to the commit log and the memtable; memtables flush to immutable SSTables and compaction merges them later [S10]. Reads may merge several SSTables and step over tombstones, so deletes, nulls and compaction backlog slow them.

> **"How do nulls hurt?"** "Writing a null creates a tombstone; Discord saw 12 per message [S1]."

Weak: "Because it is NoSQL."

Strong: The four write steps, then why reads pay.

Q10 What is a hot partition and how do you fix it? PATTERN-ONLY

Model answer. One partition key gets more traffic than its partition can serve (DynamoDB: 3,000 RCU / 1,000 WCU [S3]). Fix with higher-cardinality composite keys, write sharding with suffixes, caching or request coalescing for reads [S2], and pre-aggregation for counters.

> **"4,500 writes/s on one key?"** "At least 5 suffixed keys; I would use 8 and sum on read, or aggregate in Redis."

Weak: "Add capacity."

Strong: The limit, the number, and a fix per read or write.

Q8 to Q10 have no dated company source in the window, so no company name. They are still common follow-ups once you say "Cassandra" or "DynamoDB" in a design round.

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 11 to 13

Q11 Do NoSQL databases support transactions? PATTERN-ONLY

Model answer. Many do, with limits: DynamoDB up to 100 items and 4 MB [S4]; MongoDB multi-document on replica sets and sharded clusters [S8]; Cassandra lightweight transactions per partition with Paxos [S12], Accord multi-partition transactions in 6.0 (pre-release in September 2026) [S13].

> **"Then why not always NoSQL?"** "Limits and costs differ; joins, constraints and ad-hoc queries are still easiest in relational."

Weak: "No."

Strong: Store by store, with the limits.

Q12 Key-value store vs document store? PATTERN-ONLY

Model answer. A key-value store treats the value as opaque and finds it only by key. A document store understands fields: it indexes and queries them and updates parts of a document. DynamoDB sits between: key-based access with document-shaped items and secondary indexes.

> **"Where does Redis sit?"** "Key-value, but with typed values (hashes, sorted sets) that support some operations inside the value [S15]."

Weak: "Same thing."

Strong: Opaque value vs queryable fields, with an example.

Q13 What is polyglot persistence and what does it cost? PATTERN-ONLY

Model answer. Several store types in one product, each for the access pattern it serves best. Costs: more systems to operate and secure, data duplicated across stores, and keeping them consistent. Use one source of truth per fact and an outbox or CDC instead of dual writes.

> **"When would you not do it?"** "At small scale: PostgreSQL with jsonb and partitioned tables, plus Redis, covers most needs."

Weak: "Use the best database for everything."

Strong: Benefits, costs, the sync mechanism, and when not to.

TEMPLATE: "WHY DID YOU CHOOSE THIS DATABASE?" (60 SECONDS)

1. Pattern: "The hot path was ___, by ___ key, about ___ per second." **2. Need:** "It needed ___ (transactions / flexible fields / cheap appends)." **3. Choice:** "So I used ___, which is good at exactly that." **4. Cost:** "I gave up ___ and handled it by ___." **5. Switch point:** "I would change if ___."

INTERVIEW MODE: SAY IT LIKE THIS

- "Expect 'why this database?' about your own project, 'SQL or NoSQL and why?' inside a design round, and follow-ups on keys, hot partitions, transactions and keeping several stores in sync."

HOW TO THINK WHEN IT BREAKS

Scenario questions 1 to 4

All Pattern-only. Numbers are the lesson's assumptions unless cited.

S1. During an IPL final, one DynamoDB key throws ThrottlingException while the table is far from its total capacity. PATTERN-ONLY

1. A hot partition: one key is above 1,000 WCU or 3,000 RCU [S3].
2. Shard the key with suffixes, cache reads,
3. or aggregate writes in Redis and flush once a second.

S2. A MongoDB write fails because a restaurant document is too large. PATTERN-ONLY

1. Something unbounded is embedded (reviews, order history).
2. Move it to its own collection referencing restaurant_id,
3. with an index on (restaurant_id, created_at) [S7, S8].

S3. One chat channel takes 20 seconds to load after a moderator deleted most of its messages. PATTERN-ONLY

1. Tombstones: reads step over every deleted row until compaction.
2. Shorten the tombstone lifetime where repairs allow it; run compaction.
3. Skip empty buckets, as Discord did [S1].

S4. Orders are in Cassandra keyed by order_id; product now wants "all orders from this restaurant last week, by status". PATTERN-ONLY

1. The partition key does not match the query, so it scans.
2. Add a second table partitioned by (restaurant_id, day), or send reporting to an analytics store.
3. And ask whether orders belonged in a relational database at all.

SYMPTOM	FIRST SUSPECT	CONCEPT
Throttling on one key, table not full	Hot partition	8
Document too large	Unbounded embedded array	5
Slow reads after mass deletes	Tombstones	7
New query scans the whole table	Partition key does not match the query	6
Two stores disagree	Dual write	9
Data gone after restart	Redis persistence off	4
Stale read via an index	GSI is eventually consistent	4

HOW TO THINK WHEN IT BREAKS

Scenario questions 5 to 7

S5. An order shows "delivered" in PostgreSQL, but tracking shows "picked up". PATTERN-ONLY

1. A dual write failed or ran out of order.
2. Make PostgreSQL the source of truth; publish status changes through an outbox or CDC.
3. Make the consumer idempotent and ordered per order.

S6. After a Redis restart, users find their carts empty. PATTERN-ONLY

1. Redis ran without persistence.
2. Turn on AOF (everysec loses at most about a second [S14]) and add a replica,
3. or keep carts for logged-in users in a durable store.

S7. Right after a user updates their email, a lookup by email through a DynamoDB GSI returns the old item. PATTERN-ONLY

1. GSI reads are eventually consistent [S5].
2. Read the base table by primary key right after the write,
3. or accept a short delay for lookups by email.

Just for fun. Every "the database is slow" ticket is secretly "we queried it in a way it was never designed for". The database is not slow. It is offended.

WHAT GOES WRONG

Fixing the symptom, not the pattern. Raising capacity for a hot key, or raising the document size limit in your head. If the access pattern does not match the store, the incident comes back on the next big day.

A five-step checklist for any datastore incident

1. **Which query?** Find the exact access pattern that got slow or wrong.
2. **Does it start with the key** the store was designed for (partition key, document id)?
3. **Is one key hot?** Compare that key's rate with the per-partition limit.
4. **Is something growing without bound?** A document, a partition, a tombstone pile.
5. **Is this store the source of truth** for that fact, or a copy that can lag?

INTERVIEW MODE: SAY IT LIKE THIS

- "For throttling on one key I suspect a hot partition; for oversized documents, an unbounded array; for slow reads after deletes, tombstones; for disagreeing stores, a dual write. Each has a specific fix."

SOLVE THESE BEFORE YOU LOOK

Practice problems 1 to 3

Eight original problems, about 45 minutes. Numbers marked "assumption" are given to you. Every calculation was checked by running it when this PDF was built.

P1 A campus chat app, sized EASY

CampusChat (assumption): 5 million daily active students, each sending 40 messages a day, 200 bytes per message including metadata. (a) Messages per day and raw storage per day? (b) Storage per day with replication factor 3? (c) Which store family, and what primary key for "open a group chat, show the latest 50 messages"?

> **Hint.** Multiply; then think about the doodhwala's notebook.

SOLUTION

(a) 5,000,000 x 40 = **200 million messages a day**; x 200 B = **40 GB a day**. (b) x 3 = **120 GB a day**. (c) Wide-column: PRIMARY KEY ((group_id, bucket), message_id), message_id time-sortable and clustered newest first; bucket by time (say 10 days, as Discord did) so a busy group's partition stays bounded.

P2 DynamoDB capacity EASY

A profile item is 6 KB. At peak the app does 1,200 strongly consistent reads and 300 writes per second; each write is 2.5 KB. (a) RCU and WCU needed? (b) With eventually consistent reads? (c) Could all 1,200 strongly consistent reads per second go to one single item without hitting the per-partition limit? What about 2,000?

> **Hint.** Round item size up to 4 KB steps for reads and 1 KB steps for writes.

SOLUTION

(a) Reads: 6 KB → 2 steps of 4 KB, so 2 RCU each: 1,200 x 2 = **2,400 RCU**. Writes: 2.5 KB → 3 WCU each: 300 x 3 = **900 WCU**. (b) Half the read cost: **1,200 RCU**. (c) 2,400 RCU ≤ 3,000, so yes. 2,000 x 2 = 4,000 RCU > 3,000, so no: cache it, or read eventually consistent (2,000 RCU).

P3 Embed or reference? MEDIUM

A college's MongoDB "course" document holds the syllabus (about 40 KB). Someone proposes embedding every student's discussion post. A popular course gets 9,000 posts a semester (6 months) at about 500 B each. (a) How fast does the document grow per month? (b) Roughly when does it hit 16 MiB (16,777,216 bytes)? (c) What design do you propose?

> **Hint.** Start size plus monthly growth times months.

SOLUTION

(a) 9,000 x 500 B / 6 = **750,000 B a month**. (b) (16,777,216 - 40,000) / 750,000 ≈ **22.3 months**, in about the fourth semester; long before that every syllabus read carries megabytes of posts. (c) Posts in their own collection with course_id and created_at, indexed on (course_id, created_at); the course document embeds only the syllabus and perhaps the 5 newest post titles.

SOLVE THESE BEFORE YOU LOOK

Practice problems 4 to 6

P4 Proctoring events: the partition key MEDIUM

An online exam platform logs a proctoring snapshot event every 10 seconds per student. One national mock test: 50,000 students, 3 hours, 120 bytes per event. (a) Total events and raw data for the exam? (b) Compare partition keys exam_id, (exam_id, student_id) and (exam_id, minute): rows per partition and the problem with each. (c) Peak writes per second?

> **Hint.** 360 events per student per hour.

SOLUTION

(a) 50,000 x 360 x 3 = **54 million events**; x 120 B = **6.48 GB**. (b) **exam_id**: all 54 million rows (6.48 GB) in one partition taking every write. **(exam_id, student_id)**: 1,080 rows, about 130 KB, spread over 50,000 partitions, and it matches "show this student's timeline". **(exam_id, minute)**: 180 partitions of 300,000 events each (36 MB), and the current minute takes every write, so it is always hot. Choose (exam_id, student_id), clustered by ts. (c) 50,000 / 10 = **5,000 writes per second**.

P5 Match the store (a UPI-style payments app) MEDIUM

Pick a store family and give one reason. (a) Transfer money between two wallets. (b) OTP and "5 attempts per 10 minutes" limits. (c) A user's transaction history, newest first, 20 at a time. (d) Merchant profile pages whose fields differ by type (kirana, pharmacy, restaurant). (e) "Recent payees" on the home screen. (f) "Is this new payee two hops from a known fraud account?"

> **Hint.** Correctness first, then key shape.

SOLUTION

(a) **Relational**: both balances change in one ACID transaction. (b) **Key-value (Redis)**: a key per user with TTL; INCR and expire. (c) **Wide-column**: partition user_id (bucketed by month), clustering txn_time DESC; or a relational table indexed on (user_id, txn_time) at modest scale. (d) **Document**: fields vary by type, read whole. (e) **Key-value**: a small list per user (Redis, or DynamoDB if it must survive restarts). (f) **Graph database** (named only), or a precomputed risk score.

P6 Shard the hot key? EASY

A live-score item for a cricket final is updated 2,600 times a second (items under 1 KB) and read 45,000 times a second (eventually consistent, under 4 KB). Per partition: 1,000 WCU and 6,000 eventually consistent reads per second. (a) Minimum number of write shards? (b) Can the reads be served from DynamoDB alone on one key? (c) What would you do?

> **Hint.** Divide by the per-partition limit and round up.

SOLUTION

(a) 2,600 / 1,000 = 2.6, so **3 shards** minimum. (b) 45,000 / 6,000 = **7.5 times** one partition, so no. (c) Aggregate: the score changes far less often than 2,600 times a second, so write one score item a few times a second; readers get it from a cache or CDN with a 1-second TTL (Lesson 6). No sharding needed once writes are aggregated.

SOLVE THESE BEFORE YOU LOOK

Practice problems 7 and 8

P7 Tombstones from nulls EASY

A message table has 20 columns; a typical message sets 5. The app writes every column, sending null for the empty ones. Peak is 8,000 messages a second. (a) Tombstones per message and per second? (b) Per day at an average of 2,500 messages a second? (c) Fix?

> **Hint.** Each null written is a tombstone [S1].

SOLUTION

(a) 20 - 5 = **15 per message**; 15 x 8,000 = **120,000 per second**. (b) 15 x 2,500 x 86,400 = **3.24 billion a day**. (c) Write only the columns that have values (or use defaults), as Discord did.

P8 The dual-write gap MEDIUM

A ride app writes each trip to PostgreSQL and then, from the same request, writes a "trip started" row to Cassandra for live tracking. 1.2 million trips a day; the second write fails 0.05% of the time and is not retried. (a) Trips a day missing from tracking? (b) Over 30 days? (c) A design where this cannot happen silently?

> **Hint.** Multiply, then think "one transaction, then publish".

SOLUTION

(a) 1,200,000 x 0.0005 = **600 trips a day**. (b) 600 x 30 = **18,000**. (c) Insert the trip and an outbox row in the same PostgreSQL transaction; a relay (or CDC) writes Cassandra with retries; the write is an idempotent upsert keyed by trip_id, so retries are safe; alert if the outbox backlog grows.

HOW TO CHECK YOUR OWN STORE ANSWER

Write the access pattern first (query, key, rate, size, correctness). Name the family and the key. Compute rows and bytes per partition for the worst key holder. Check the hottest key against the per-partition limit. Say what is the source of truth, and how the other stores get the data.

INTERVIEW MODE: SAY IT LIKE THIS

- "For a datastore question with numbers I size rows per day, bytes, replication and peak writes; compute rows and bytes per partition for each candidate key; convert item sizes to capacity units; and check every key against per-partition limits and the worst day."

Problems 4 and 8 use a mock-test and a ride app on purpose: the same reasoning works for any product. The pair task in class uses Swiggy-style features.

Just for fun. If your answer to every practice problem was "MongoDB", please re-read Concept 2. Slowly. With chai.

EIGHT QUESTIONS, TWO MINUTES EACH

Mini quiz

Original questions. Cover the answers, answer all eight, then check. The answer letters are spread: two each of A, B, C and D.

1. Which access pattern most strongly suggests a wide-column store?

A) Transfer money between two accounts
B) Get a session by id
C) Append millions of GPS pings; read one partner's pings for today in time order
D) Full-text search over menus

Answer: C. A keyed, time-ordered, append-heavy stream.

2. A DynamoDB item is 9 KB. How many RCU does one strongly consistent read use?

A) 3 **B)** 2 **C)** 9 **D)** 1

Answer: A. 9 KB rounds up to 3 steps of 4 KB.

3. Which is the best reason to embed data in a MongoDB document?

A) The child data grows without limit
B) The child is often queried on its own
C) The child changes often and is duplicated in many parents
D) It is a "contains" relationship read together with the parent

Answer: D. MongoDB's embedding guidance [S8].

4. In PRIMARY KEY ((channel_id, bucket), message_id), what does message_id do?

A) Decides which node stores the row
B) Orders rows inside the partition
C) Is a secondary index **D)** Sets the replication factor

Answer: B. It is the clustering key.

5. Redis with appendfsync everysec crashes. What can be lost?

A) Nothing **B)** About one second of writes
C) Everything since the last daily snapshot
D) Only reads

Answer: B. Redis docs [S14].

6. Which statement about transactions in October 2026 is correct?

A) DynamoDB transactions can include up to 100 items
B) MongoDB has no multi-document transactions
C) Cassandra 5.0 has multi-partition ACID transactions by default
D) Redis transactions are always durable

Answer: A. TransactWriteItems: 100 items, 4 MB [S4].

7. A partition key of (city, day) for delivery-partner pings is bad mainly because:

A) City names are long **B)** It needs a sort key
C) It is too random
D) One busy city's partition becomes huge and hot

Answer: D. Bengaluru alone: 21.6 GB.

8. dabba.app writes orders to PostgreSQL, then tracking events to Cassandra from the same request. The safest fix for missing events is:

A) Write to Cassandra first
B) Retry forever inside the request
C) An outbox row in the same PostgreSQL transaction, published by a relay or CDC
D) Use Redis instead

Answer: C. One transaction, then publish.

Just for fun. You can now answer "why MongoDB?" with something better than "because the tutorial used it". Your interviewer will notice. Your tutorial will not.

SEVEN SENTENCES TO KEEP

Recap and the 60-second answer

1. "SQL or NoSQL" is the wrong first question. List the access patterns: query, key, rate, read/write mix, size, correctness.
2. Relational is the default: joins, constraints and multi-row transactions; it scales with replicas, sharding and distributed SQL.
3. Key-value stores serve exact-key lookups at high rate: Redis in memory (sub-millisecond, persistence optional), DynamoDB durable with per-partition limits of 3,000 RCU and 1,000 WCU.
4. Document stores serve whole entities with varying nested parts; embed what is read together, reference what grows or stands alone; mind the 16 MiB limit.
5. Wide-column stores serve append-heavy data read as rows for one key in order; the partition key decides the node, the clustering key the order; bound partitions; deletes leave tombstones.
6. Hot keys come back as hot partitions in every store: high-cardinality keys, write sharding, caching and coalescing.
7. Polyglot persistence gives each access pattern its store, at the cost of more systems; one source of truth per fact, synced by outbox or CDC, not dual writes.

SAY THIS IN AN INTERVIEW: "WHICH DATABASE WOULD YOU USE?"

"I choose a datastore by listing access patterns, default to relational for joins and money, use key-value for exact-key lookups, documents for whole entities with varying parts, and wide-column for append-heavy keyed streams, then check partition keys, limits and sizes, and keep one source of truth per fact with changes flowing by events or CDC."

MEMORY HOOK: THE FAMILIES

Token, **folder**, **register**, **notebook**: key-value, document, relational, wide-column.

MEMORY HOOK: THE RULE

Pattern first, relational by default, **one truth** per fact.

EARLIER LESSON	HOW THIS LESSON USED IT
Lesson 3: access patterns, transactions	The access-pattern list picks the store; relational stays the default for joins and money
Lesson 4: estimation	Rows per day, bytes, partition sizes, capacity units
Lesson 6: Redis, hot keys	Redis as a key-value store; caches in front of hot partitions
Lesson 7: shard keys, consistent hashing	Partition keys are shard keys built into the store; salting = write sharding

Next lesson: Lesson 9 states CAP properly and explains consistency models and quorums, which is what "eventually consistent read" and "quorum write" in this lesson actually mean.

CHECK OUR WORK

Sources (1 of 2)

Every [S#] is a page opened in this session. Every calculation in this lesson was recomputed in code when this PDF was built.

ID	PAGE	WHAT IT SUPPORTED	URL
S1	Discord, S. Vishnevskiy, "How Discord Stores Billions of Messages" (13 Jan 2017)	MongoDB replica set; data and index beyond RAM at ~100M messages; 50/50 random reads; Cassandra ((channel_id, bucket), message_id), 10-day buckets, <100 MB; 12 nodes RF 3; 12 null tombstones; 20 s load; 120M messages/day	discord.com/blog/how-discord-stores-billions-of-messages
S2	Discord, B. Ingram, "How Discord Stores Trillions of Messages" (6 Mar 2023)	177 Cassandra nodes; hot partitions, GC, compaction; 72 ScyllaDB nodes; p99 40–125 ms → 15 ms; request coalescing; 3.2M msgs/s migration	discord.com/blog/how-discord-stores-trillions-of-messages
S3	AWS DynamoDB Developer Guide: partition key best practices	3,000 read units and 1,000 write units per partition; 20 KB item example	docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key-design.html
S4	AWS DynamoDB Developer Guide: Constraints	400 KB item; transactions up to 100 items and 4 MB	docs.aws.amazon.com/amazondynamodb/latest/developerguide/Constraints.html
S5	AWS DynamoDB Developer Guide: Global secondary indexes	GSI reads eventually consistent	docs.aws.amazon.com/amazondynamodb/latest/developerguide/GSI.html
S6	AWS DynamoDB Developer Guide: Cheat sheet	RCU 4 KB (strong or two eventual), WCU 1 KB	docs.aws.amazon.com/amazondynamodb/latest/developerguide/CheatSheet.html
S7	MongoDB Manual: Limits and Thresholds	BSON document maximum 16 mebibytes	mongodb.com/docs/manual/reference/limits/
S8	MongoDB Manual: Best Practices for Data Modeling	Embed vs reference; schema validation; distributed transactions on replica sets and sharded clusters	mongodb.com/docs/manual/data-modeling/best-practices/
S9	Apache Cassandra docs: Data modeling introduction	Query-driven modelling; partition key; consistent-hashing variant	cassandra.apache.org/doc/latest/cassandra/developing/data-modeling/intro.html
S10	Apache Cassandra docs: Storage Engine	Commit log, memtable, flush, SSTables	cassandra.apache.org/doc/stable/cassandra/architecture/storage-engine.html
S11	Apache Cassandra, "Announcing Apache Cassandra 5.0" (5 Sep 2024) and SAI docs	5.0 GA; storage-attached indexes	cassandra.apache.org/_/blog/Apache-Cassandra-5.0-Announcement.html
S12	DataStax docs: Lightweight transactions	IF clauses; Paxos compare-and-set; four round trips	docs.datastax.com/en/cassandra-oss/2.1/cassandra/dml/dml_ltwt_transaction_c.html
S13	Digitalis, "Every change in Apache Cassandra 6.0" (14 Sep 2026)	6.0-alpha3; 5.0.9 current GA; Accord beta, off by default	digitalis.io/post/apache-cassandra-6-0-everything-that-changed-in-one-page
S14	Redis docs: Persistence	RDB and AOF; everysec may lose 1 second	redis.io/docs/latest/operate/oss_and_stack/management/persistence/
S15	Redis docs: Data types, sorted sets	Types list; leaderboards; sliding-window rate limiters	redis.io/docs/latest/develop/data-types/sorted-sets/
S16	PostgreSQL 18 docs: 8.14 JSON Types	jsonb binary format, faster to process; GIN indexing	postgresql.org/docs/current/datatype-json.html

CHECK OUR WORK

Sources (2 of 2)

ID	PAGE	WHAT IT SUPPORTED	URL
S17	AWS Database Blog, Zomato billing platform from TiDB to DynamoDB (22 Dec 2023)	Why TiDB was replaced; single table; composite key; GSI suffix; ~90% lower latency (~75 ms), 2,000 → 8,000 req/min, ~50% lower cost	aws.amazon.com/blogs/database/unlocking-performance-scalability-and-cost-efficiency-of-zomatos-billing-platform-by-switching-from-tidb-to-dynamodb
S18	Swiggy Bytes, S. Mangalgi, "Building a mind reader at Swiggy using Data Science" (30 Jan 2023)	Swiggy Suggests precomputed per customer-geohash-meal slot in DynamoDB	bytes.swiggy.com/building-a-mind-reader-at-swiggy-using-data-science-5a5c38aa6c17
S19	Swiggy Bytes, A. Garg, "Live Data Transfer: RDS to DynamoDB Made Easy" (Jul 2024)	Auth platform RDBMS → DynamoDB for elasticity; live migration	bytes.swiggy.com/live-data-transfer-rds-to-dynamodb-made-easy-2f2aef94f847
S20	PingCAP blog (vendor), "Why Flipkart Chose TiDB to Replace Its Large MySQL Fleet" (3 Apr 2023)	700+ MySQL clusters; manual sharding pain; SQL and ACID kept	pingcap.com/blog/why-flipkart-chose-tidb-to-replace-its-large-mysql-fleet/
S21	Uber Blog, "Evolving Schemaless into a Distributed SQL Database" (23 Feb 2021)	Schemaless history; Docstore on MySQL; schemas; transactions	uber.com/blog/schemaless-sql-database/
S22	Netflix TechBlog, "Introducing Netflix's Key-Value Data Abstraction Layer" (18 Sep 2024)	Cassandra use cases; datastore misuse	netflixtechblog.com/introducing-netflixs-key-value-data-abstraction-layer-1ea8a0a11b30
S23	AWS News Blog, C. Yun, "All the numbers: Amazon Prime Day 2026 powered by AWS" (6 Oct 2026)	DynamoDB 59 trillion requests, peak 192M/s, single-digit ms	aws.amazon.com/blogs/aws/all-the-numbers-amazon-prime-day-2026-powered-by-aws/
S24	LeetCode Discuss, "Eternal Ltd (Zomato) SDE-1 Interview Experience" (23 Jul 2025)	Postgres; why not MongoDB; when Postgres; ACID; CAP	leetcode.com/discuss/post/6994052/eternal-ltdzomato-sde-1-interview-experi-fbnm/
S25	Medium, "My Zomato SDE Intern Interview Experience (6-Month Internship)" (16 Apr 2026)	Why PostgreSQL; both PostgreSQL and MongoDB; transactions; ACID	medium.com/@prachisah66614/my-zomato-sde-intern-interview-experience-6-month-internship-9b062ada4c91
S26	interviewexperiences.in, "Swiggy SDE II Interview Experience" (12 Jun 2025; interview Nov 2024)	SQL vs NoSQL differences, use cases, pros and cons	interviewexperiences.in/experience/swiggy/swiggy-sde-ii-interview-experience-java-spring-boot
S27	LeetCode Discuss, "Uber SDE-2 Interview Experience, L4, Bangalore" (12 Sep 2025)	Uber Eats homepage; schema; SQL vs NoSQL justification	leetcode.com/discuss/post/7182332/uber-sde-2-interview-experience-l4-banga-fd1d
S28	LeetCode Discuss, "Amazon SDE-2, 3 YOE, Accepted offer" (8 Jan 2026)	Auction; Redis sorted sets; Redis not source of truth; inventory on NoSQL	leetcode.com/discuss/post/7477684/amazon-india-sde-2-interview-experience-ca6y0
S29	Roundz newsletter, "Interview Experience 55: Amazon SDE II L5" (2 May 2025)	Blogging platform; choice of database	roundz.substack.com/p/interview-experience-55-amazon-sde2-l5
S30	AmbitionBox, Infosys System Engineer interview questions (interview Jan 2026)	"Why did you choose MongoDB over MySQL?"	ambitionbox.com/interviews/infosys-interview-questions/system-engineer
S31	Uber Blog, "How Uber Serves Over 40 Million Reads Per Second…" (15 Feb 2024)	Docstore: tens of PBs, tens of millions of requests/s	uber.com/blog/how-uber-serves-over-40-million-reads-per-second-using-an-integrated-cache/

INTERVIEW MODE: SAY IT LIKE THIS

- "Store behaviour comes from the DynamoDB, MongoDB, Cassandra, Redis and PostgreSQL documentation; real-world choices from Discord, Zomato, Swiggy, Uber, Netflix and AWS posts; interview evidence from six dated reports and one aggregator listing."

DID WE COVER THE CHECKLIST?

Coverage map and honest gaps

CHECKLIST ITEM	COVERED WHERE
What each store type is good and bad at	Concepts 1 and 3, Diagram 1, cheat sheet, misconceptions, Q3, Q12
Access patterns decide the store	Hook, Concept 2, worked example Step 1, Diagram 1, Q1, Q2, Problem 5
Key-value (Redis, DynamoDB) use cases	Concepts 4 and 8, Q5, Q10, scenarios S1, S6, S7, Problems 2 and 6
Document stores (MongoDB) use cases	Concept 5, Q2, Q6, Q7, scenario S2, Problem 3
Wide-column (Cassandra) use cases	Hook, Concepts 6 and 7, Diagram 2, Q8, Q9, scenarios S3, S4, Problems 1, 4 and 7
Polyglot persistence in one product	Concept 9, worked example, Diagram 3, Q4, Q13, scenario S5, Problem 8

Touched lightly or left out (on purpose)

- Graph databases and search engines: named only, as two more families. Search returns in the design lessons.
- CAP, consistency levels, quorum reads and writes (what "eventually consistent read" means precisely): Lesson 9.
- Queues, event streams, the outbox pattern and CDC in depth: Lesson 10.
- Columnar analytics warehouses (BigQuery, Redshift, ClickHouse) differ from wide-column stores despite the similar name; not covered here.
- Vendor licensing changes (for example Redis's licence history and the Valkey fork) are left out; check current terms before choosing.
- Cassandra 6.0 and Accord were pre-release in the September 2026 source; re-check before quoting them as generally available.
- Company frequency: six dated reports in the window; Cassandra internals and DynamoDB capacity questions are Pattern-only.

WHERE THE OPEN THREADS GO NEXT

Lesson 9 explains what "eventually consistent read", "strongly consistent read" and "quorum" mean, which you used here for DynamoDB GSIs and Cassandra replicas. **Lesson 10** builds the queue and event stream behind the outbox and CDC arrows in the dabba.app diagram.

INTERVIEW MODE: SAY IT LIKE THIS

- "This lesson covers the four store families, access-pattern-driven choice, key-value, document and wide-column use cases, and polyglot persistence. Consistency models follow in Lesson 9 and event streams in Lesson 10."

SELF-CHECK

Can you say yes to all four?

- **I can list access patterns**
  Query, key, frequency, mix, size, correctness, then the tree.
- **I can size a key-value table**
  RCU, WCU and the per-partition limit.
- **I can design a wide-column key**
  Partition = question, clustering = order, bucket = bound.
- **I can defend a polyglot layer**
  One job and one source of truth per store, no dual writes.

> "Discord's query never changed; the database did, twice." **Now you can say why, and pick the store for any feature from its access pattern, with a number for every box.**

YOUR NEXT STEPS

STEP 01
Solve problems 1 to 8 on paper

STEP 02
Write the access-pattern list for your GenAI project

STEP 03
Read ahead: Lesson 9, CAP and quorums